NANYANG TECHNOLOGICAL UNIVERSITY

CCDS25-0356: GENERATIVE AI (GPT) + QUANT FINANCE FOR WEALTH MANAGEMENT

Tran Khanh Linh

College of Computing and Data Science

AY2025-26

NANYANG TECHNOLOGICAL UNIVERSITY

CCDS25-0356: GENERATIVE AI (GPT) + QUANT FINANCE FOR WEALTH MANAGEMENT

Submitted in Partial Fulfilment of the Requirements
for the Degree of Bachelor of Computing (Hons) in Data Science and Artificial Intelligence
of the Nanyang Technological University

by

Tran Khanh Linh

College of Computing and Data Science

AY2025-26

Abstract

This project designs and implements an AI-assisted Exchange-Traded Fund (ETF) recommendation platform. Its goal is to make the complex ETF market more accessible for retail investors. The platform uses a rule-based multi-factor recommendation engine, an AI-powered chat interface for educational support, news sentiment analysis, and risk management tools including stress scenario analysis and risk measures like VaR and CVaR.

Built with Next.js, FastAPI, and PostgreSQL, the system supports end-to-end workflows. This includes user authentication, risk profiling through questionnaires, ETF discovery, delivery of personalized recommendations, and wallet-enabled portfolio management. The data layer includes U.S. and Singapore ETF universes, complete with historical market records and related information to support clear analytics.

The results indicate that combining transparent scoring logic with AI-assisted interaction can effectively support non-expert investors in making decisions while keeping the focus educational and non-advisory. This work offers a reference architecture that can be deployed to integrate financial analytics, clear recommendations, and user-focused AI support in retail investment platforms.

Acknowledgments

I am grateful for the opportunity to get involved in and further explore this project, as it allows me to delve more deeply into the use of LLMs in quantitative finance.

I would also like to express my gratitude to Assoc Prof Wee Keong NG for granting me this opportunity. His continual support and endorsement throughout this project have been immensely helpful to me.

I would like to extend my heartfelt thanks to my friends, who generously tested the products and provided valuable feedback. Their insights, suggestions, and encouragement have been crucial in improving the usability and robustness of this project.

Finally, I am deeply thankful to the open-source community, which has provided me with many ideas for the architecture and technologies that I have used in this project.

Contents

List of Tables

List of Figures

Chapter 1

Introduction

1.1 Background

1.1.1 Overview of ETFs

An exchange-traded fund (ETF) is an investment fund that holds multiple underlying assets such as equities, bonds, commodities, or derivatives and trades on an exchange, much like an individual stock [1, 2]. ETFs can be structured to track anything from the price of a commodity to a large and diverse collection of stocks, or even specific investment strategies.

In recent years, ETFs have emerged as one of the most popular investment vehicles for both retail and institutional investors. As of 2025, the global ETF market has surpassed $12 trillion in assets under management, with over 10,000 ETFs available worldwide [3]. ETFs offer several advantages over traditional mutual funds, including lower expense ratios, intraday trading flexibility, tax efficiency, and transparent holdings disclosure. Compared to individual stocks, ETFs also provide immediate diversification, reducing unsystematic risk while requiring less capital and expertise from investors.

There are various types of ETFs, differentiated by asset class (e.g., equities, bonds, commodities, currencies), investment strategy (passive index tracking, active management, smart beta), sector or theme (technology, healthcare, clean energy), market focus (international, emerging markets), and complexity (leveraged, inverse, synthetic) [1]. For example, an index ETF aims to replicate the performance of a specific benchmark such as the S&P 500, while an active ETF is managed by professionals seeking to outperform a benchmark. These ETF types are further explained in Chapter 2.

1.1.2 Challenges Faced by Retail Investors

Despite the rapid expansion and increased accessibility of ETFs, retail investors continue to face several challenges. The large number of ETFs spanning multiple asset classes, geographical regions, and investment strategies can lead to information overload, making it difficult for investors to identify funds that align with their financial goals and risk preferences [4].

Effective ETF selection also requires an understanding of risk metrics such as beta, volatility, and the Sharpe ratio. However, many retail investors lack the financial literacy needed to accurately interpret these measures, resulting in portfolios that may not reflect their true risk tolerance [5]. Furthermore, ETF performance is influenced by macroeconomic trends, policy decisions, and market news, which retail investors may struggle to interpret and incorporate into timely investment decisions [6].

These challenges are compounded by limited access to professional-grade analytical platforms such as Bloomberg Terminal and Morningstar Direct, which are expensive and primarily designed for institutional users [6]. As a result, retail investors often rely on simplified tools with limited analytical depth.

1.2 Motivation

Recent advances in Artificial Intelligence (AI) and Large Language Models (LLMs) have created new opportunities to democratise access to financial insights. AI-powered systems can process large volumes of financial data, support personalised recommendations, explain complex concepts in plain language, and analyse news sentiment in real time. In this project, AI is used as an educational decision-support layer, while recommendations are generated through transparent rule-based risk and metric screening rather than autonomous investment advice. By combining these capabilities, the project aims to bridge the gap between professional-grade investment tools and the practical needs of retail investors.

1.3 Objectives

The primary objective of this project is to design and implement an AI-assisted ETF recommendation platform that helps retail investors build and manage diversified portfolios aligned with their risk preferences. The specific objectives of the project are as follows:

- Develop a rule-based multi-factor recommendation engine incorporating risk, performance, cost, and liquidity metrics with explainable scoring.
- Integrate an AI-powered chatbot to explain ETFs, financial concepts, and recommendations in accessible language.
- Aggregate and manage ETF market data and compute key financial and risk metrics on demand.
- Implement risk analysis tools, including historical stress testing and Value at Risk (VaR).
- Support wallet-level portfolio organization and profiling for goal-based investment planning.
- Build a responsive and user-friendly web interface for ETF discovery, portfolio management, and visualization.
- Evaluate system functionality and usability through comprehensive testing and user feedback.

1.4 Scope

The scope of this project includes the following components:

- **Market coverage:** 2,272 ETFs listed on U.S. exchanges and the Singapore Exchange.
- **Historical data:** Up to 15 years of daily price data (2010–2025) covering multiple market cycles.
- **Macroeconomic data:** Key indicators including CPI, GDP, interest rates, Treasury yields, and VIX.
- **News and sentiment:** ETF-related financial news with sentiment analysis, prioritised by assets under management.
- **Core features:** User authentication, risk profiling, ETF discovery and analysis, wallet-based portfolio management, rule-based multi-factor recommendations, AI chatbot, news sentiment analysis, and risk scenario analysis (stress testing and VaR).
- **Technology stack:** React/Next.js frontend, FastAPI backend, PostgreSQL (Supabase) database, deployed on Vercel and Render.
- **Target users:** Retail investors with basic to moderate financial knowledge seeking ETF investment insights.

1.5 Report Organisation

This report is organised into the following chapters:

- **Chapter 1** presents the background, motivation, objectives, scope, and organisation of the project.
- **Chapter 2** reviews relevant technical concepts including ETFs, risk metrics, and AI/ML applications in finance, and examines existing systems.
- **Chapter 3** describes the system requirements, architecture, database design, API design, and user interface design with supporting diagrams.
- **Chapter 4** details the technology stack, backend and frontend implementation, key algorithms, and integration of AI components.

- **Chapter 5** demonstrates the completed platform through its core user workflows and presents the evaluation results.
- **Chapter 6** summarises the project outcomes, discusses limitations, and proposes recommendations for future work.

Chapter 2

Literature Review

2.1 Exchange-Traded Funds and Market Structure

Exchange-Traded Funds (ETFs) represent one of the most significant financial innovations of the last three decades. An ETF is a pooled investment security that operates similarly to a mutual fund but trades on a stock exchange throughout the day at market-determined prices. The modern ETF era is widely attributed to the 1993 launch of the Standard & Poor's Depositary Receipts (SPDR), commonly known by its ticker symbol "SPY," which tracks the S&P 500 Index [7].

Since 1993, the ETF market has expanded into a diverse ecosystem that includes:

- **Equity ETFs:** Tracking broad market indices or specific market capitalizations.
- **Bond / Fixed Income ETFs:** Providing exposure to government, corporate, or municipal debt.
- **Commodity ETFs:** Tracking physical assets such as gold or oil.
- **Sector and Thematic ETFs:** Focused on specific industries (e.g., Technology) or investment themes (e.g., Clean Energy).

The structural foundation of ETFs lies in the creation and redemption mechanism managed by Authorized Participants (APs). Unlike mutual funds, which are priced only at the end of the trading day based on Net Asset Value (NAV), ETFs allow APs to exchange baskets of underlying securities for ETF shares and vice versa. This arbitrage mechanism ensures that ETF market prices remain closely aligned with their intrinsic value [1]. Compared to traditional mutual funds, ETFs generally offer lower expense ratios, improved tax efficiency, and higher liquidity, although they typically lack discretionary alpha-seeking stock selection.

2.2 Financial Performance and Risk Metrics

Evaluating financial instruments requires a multi-dimensional framework that accounts for both return and risk-adjusted performance [8].

2.2.1 Core Performance Metrics

Returns are commonly analyzed across multiple investment horizons, including Year-to-Date (YTD), 1-month, 3-month, 6-month, 1-year, 3-year, and 5-year periods, where returns refer to the percentage gain or loss in the value of an investment over a specified period. This approach enables investors to distinguish between short-term momentum and long-term performance stability. However, return analysis alone is insufficient without considering ownership costs, particularly the *Expense Ratio*, which represents the annual operating fees expressed as a percentage of assets under management (AUM), or simply the yearly cost investors pay to hold the fund.

2.2.2 Risk and Volatility

Risk can be quantified using several established statistical measures:

- **Volatility (σ):** The standard deviation of returns, measuring the dispersion of price movements.
- **Beta (β):** A measure of systematic risk relative to the broader market, commonly benchmarked against the S&P 500. A value of $\beta > 1$ indicates higher sensitivity to market fluctuations.
- **Maximum Drawdown (MDD):** The largest observed decline from a peak to a

trough, representing the worst historical loss scenario.

2.2.3 Formula Summary

Metric	Definition	Formula
Arithmetic Return	Percentage price change	$R = \frac{P_t - P_{t-1}}{P_{t-1}}$
Sharpe Ratio	Risk-adjusted return	$S = \frac{R_p - R_f}{\sigma_p}$
Beta	Sensitivity to market	$\beta = \frac{Cov(R_p, R_m)}{Var(R_m)}$
Expense Ratio	Total fund costs	$ER = \frac{\text{Total Fund Costs}}{\text{Total Fund Assets}}$

Table 2.1: Summary of Key Financial Performance and Risk Metrics

The Sharpe Ratio, originally proposed by Sharpe [9], is particularly important for recommendation systems, as it enables comparison between funds with different volatility profiles by normalizing excess return per unit of risk. In practical terms, this value indicates how much additional return an investor receives for each unit of risk taken—where a higher Sharpe Ratio suggests more efficient performance (i.e., better returns for the same level of risk), while a lower or negative value implies that the investment may not be adequately compensating for its risk.

2.3 Modern Portfolio Theory and Diversification

Modern Portfolio Theory (MPT), introduced by Harry Markowitz in 1952 [10], asserts that an investment's risk and return should be evaluated in the context of its contribution to overall portfolio performance rather than in isolation.

A central concept of MPT is the *Efficient Frontier*, which represents a set of portfolios offering the maximum expected return for a given level of risk. This framework later led to the Capital Asset Pricing Model (CAPM) developed by William Sharpe in 1964 [11], distinguishing between systematic (market-wide) risk and unsystematic (asset-specific) risk.

In the ETF context, MPT provides the theoretical justification for diversification. By combining ETFs with low or negative correlations—such as equities and commodities—investors can reduce unsystematic risk. In this project, MPT principles guide the recommendation engine to ensure that users receive diversified portfolios aligned with their risk tolerance rather than isolated high-performing funds.

2.4 Scenario Analysis and Value at Risk

Scenario analysis and stress testing complement historical performance metrics by providing forward-looking risk assessments.

Value at Risk (VaR) is a statistical technique used to estimate the maximum potential loss of a portfolio over a specified time horizon at a given confidence level [12]. Common approaches include:

- **Historical VaR:** Uses ranked historical returns, where the 95th percentile loss represents the VaR.
- **Parametric VaR:** Assumes normally distributed returns and estimates risk using mean and standard deviation.
- **Monte Carlo Simulation:** Generates thousands of simulated price paths based on historical volatility and drift.

While VaR is widely adopted in industry, it has a well-known limitation: it quantifies the threshold loss at a given confidence level but provides no information about the severity of losses that exceed that threshold. To address this, Conditional Value at Risk (CVaR), also known as Expected Shortfall (ES), was introduced as a coherent risk measure [13]. CVaR is defined as the expected loss given that the loss exceeds the VaR threshold:

$$\mathrm{CVaR}_\alpha = \mathbb{E}\big[L \mid L \geq \mathrm{VaR}_\alpha\big]$$

For example, at a 95% confidence level, VaR answers "What is the minimum loss in the worst 5% of scenarios?" whereas CVaR answers "What is the average loss across that worst 5%?" CVaR is therefore more sensitive to tail risk and is recommended by the Basel III regulatory framework for internal risk models.

In addition to VaR and CVaR, stress testing is essential to simulate historical crises such as the 2008 Global Financial Crisis or the 2020 COVID-19 market crash, allowing investors to evaluate portfolio resilience under severe conditions. In the current implementation of this project, the platform emphasizes historical scenario simulation together with historical and parametric VaR and CVaR outputs, while Monte Carlo simulation is treated as a future extension.

2.5 News and Sentiment Analysis in Finance

The Efficient Market Hypothesis (EMH) posits that asset prices reflect all publicly available information [14]. However, behavioral finance research suggests that investor sentiment influences short-term price movements.

Advances in Natural Language Processing (NLP) have enabled the quantification of financial sentiment. Techniques such as the Loughran–McDonald Financial Sentiment Dictionary [15] allow models to distinguish between general negative language and finance-specific negative terminology. Modern APIs leverage transformer-based models to assign sentiment scores ranging from bearish to bullish, providing real-time market insights.

Integrating sentiment analysis into ETF platforms addresses the lag inherent in traditional fundamental data and enables dynamic assessment of thematic trends, such as the impact of regulatory news on sector ETFs.

2.6 Recommendation Systems in Finance

Recommendation systems generally follow three architectural paradigms [16]:

- **Content-Based Filtering:** Recommends assets similar to those previously selected by the user.
- **Collaborative Filtering:** Leverages preferences of users with similar behavior.

- **Hybrid Systems:** Combines both approaches to mitigate cold-start limitations.

In finance, recommendation systems must be constrained by risk-profile alignment. A recommendation is ineffective if it violates a user's risk tolerance. Additionally, explainability is critical; users are more likely to trust systems that provide transparent, interpretable justifications for recommendations.

2.7 Large Language Models for Financial Applications

Large Language Models (LLMs) have transformed how users interact with financial data [17]. LLMs enable reasoning, summarization, and natural language explanation of complex financial concepts.

Key considerations for LLM integration include prompt engineering versus fine-tuning, mitigation of hallucination risks through Retrieval-Augmented Generation (RAG) [18], and ethical constraints to ensure compliance with financial regulations by emphasizing education rather than direct investment advice.

2.8 Related Platforms and Comparative Analysis

A comparative analysis of existing platforms highlights current limitations:

Platform	Target User	Key Strengths	Limitations
Bloomberg Terminal	Institutional	Deep financial data, real-time analytics, comprehensive market coverage	High cost (USD 24k+/year); steep learning curve
Morningstar	Professional / Retail	Trusted fund ratings and in-depth investment research	Subscription-based; relatively static interface
Betterment	Retail	Automated portfolio construction and rebalancing	Limited customization; opaque recommendation logic
ETF.com / Yahoo Finance	Retail	Free access to broad ETF and market data	Information overload; lack of AI-driven insights

Table 2.2: Comparison of Existing ETF and Investment Platforms

Existing solutions are either prohibitively expensive, overly passive, or lack integrated

AI-driven explanation. There is a clear absence of platforms that combine quantitative risk modeling with qualitative AI analysis in a single interface.

2.9 Research Gaps and Justification

This literature review identifies three key research gaps:

- **Educational Accessibility:** Abundant data exists, but retail understanding of financial risk metrics remains limited.
- **Sentiment–Quant Integration:** News sentiment is rarely linked directly to volatility and risk analysis.
- **Explainable Recommendations:** Robo-advisors seldom provide transparent, natural language justifications.

By integrating quantitative risk analytics with LLM-based explanations and sentiment analysis, this project bridges the gap between institutional-grade investment tools and retail-level accessibility.

Chapter 3

System Requirements and Design

3.1 Primary Users

The primary users of the AI-Assisted ETFs Recommendation Platform are retail investors with beginner to intermediate financial knowledge, seeking long-term wealth accumulation through diversified ETFs without frequent trading. They face challenges such as information overload, difficulty interpreting risk metrics, and limited access to personalized investment guidance, and therefore need a simplified, educational platform that provides risk-aware recommendations, clear explanations, and intuitive visualizations.

3.2 Use Cases

Figure 3.1 presents the high-level use case diagram illustrating the interactions between users and the system.

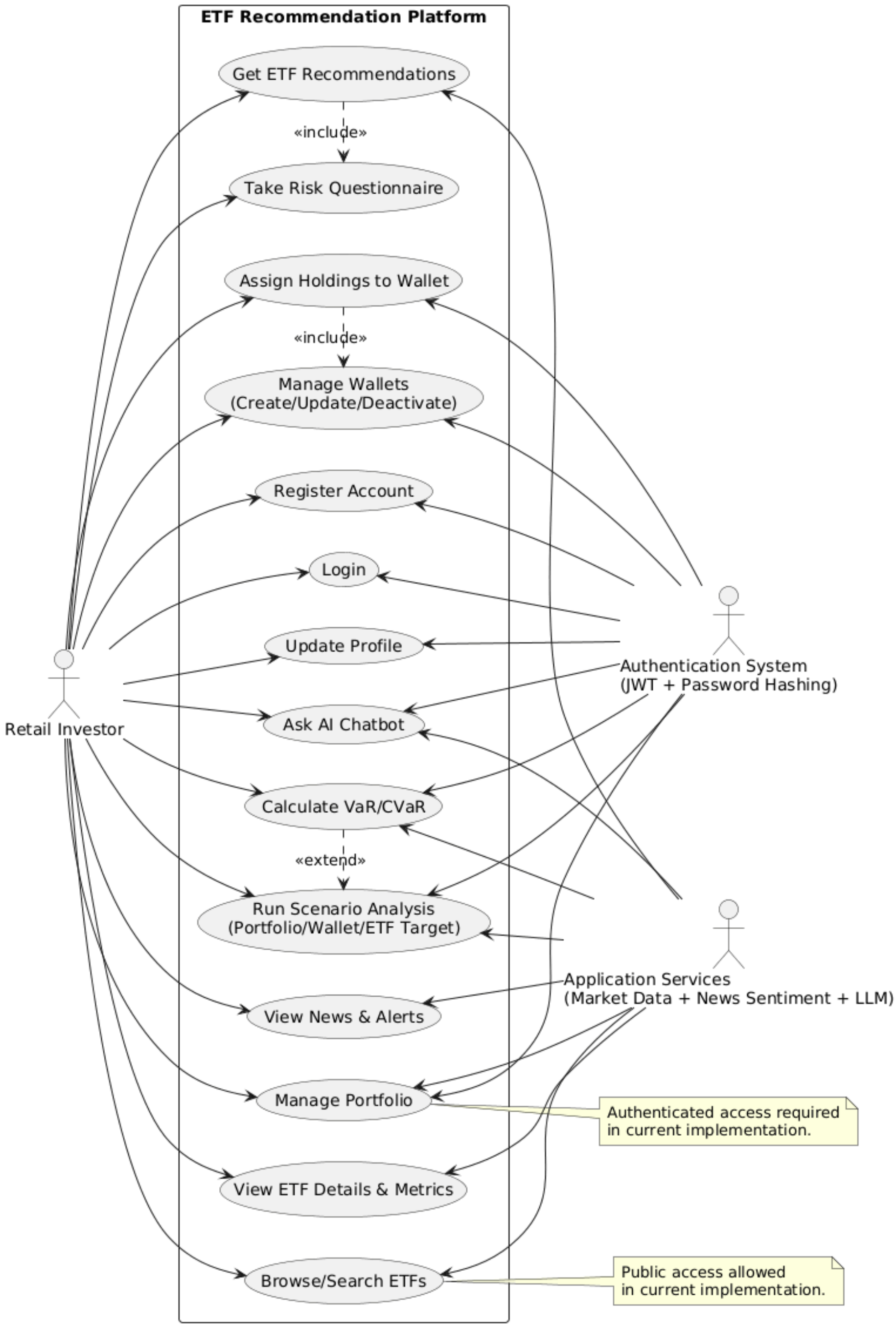

Figure 3.1: Use Case Diagram

Detailed use case specifications, including preconditions, main flows, and exception flows, are provided in **Appendix A**.

3.3 Functional Requirements

The functional requirements specify the core functionalities of the AI-Assisted ETF Recommendation Platform.

User Authentication and Profile Management

- The system shall allow users to register using an email address and password.
- The system shall allow users to log in securely to access personalized features.
- The system shall allow users to update their personal profile information.

Risk Assessment

- The system shall provide a risk profiling questionnaire to assess user risk tolerance.
- The system shall calculate a numerical risk score based on user responses.
- The system shall assign users to a predefined risk category (Conservative, Balanced, Aggressive).

ETF Discovery and Analysis

- The system shall allow users to search ETFs by ticker or name.
- The system shall allow users to filter ETFs based on asset class, region, and expense ratio.
- The system shall display detailed ETF information including historical price data and key risk metrics.

Personalized Recommendations

- The system shall generate ETF recommendations aligned with the user's risk profile.
- The system shall rank recommended ETFs using a multi-factor scoring approach.
- The system shall provide explanations for the generated recommendations.

Portfolio Management

- The system shall allow users to create and manage ETF portfolios.
- The system shall allow users to add, update, and remove holdings with optional wallet assignment.
- The system shall visualize portfolio asset allocation and performance.
- The system shall conduct risk analysis, including stress testing and Value at Risk (VaR), for portfolio, wallet, or selected ETF targets.

Wallet Management

- The system shall allow authenticated users to create, update, and deactivate wallets.
- The system shall allow users to configure wallet-level profile settings (e.g., risk profile, horizon, objective).
- The system shall display wallet-level holdings and valuation summaries.

AI Assistance and News

- The system shall provide an AI-powered chatbot to answer ETF-related queries.
- The system shall display ETF-related and portfolio-related news with sentiment analysis and alerts.
- The system shall include a financial disclaimer in all AI-generated responses.

3.4 Non-Functional Requirements

The system must meet the following non-functional requirements to ensure performance, reliability, security, and usability:

- **Performance:** Target API response time for common read requests is under 500ms under normal load, and frontend first contentful paint (FCP) target is under 1.5 seconds.

- **Scalability:** The database shall support millions of price records without significant query degradation.
- **Security:** Passwords shall be hashed; all data in transit shall be encrypted; API keys shall be stored securely on the server side.
- **Reliability:** Background tasks (e.g., data ingestion) shall handle API rate limits and retry on failure.
- **Usability:** The interface shall be responsive across mobile, tablet, and desktop devices.

3.5 System Architecture

The platform follows a modern three-tier architecture:

- **Presentation Layer (Frontend):** Built with Next.js (React), TypeScript, and Tailwind CSS for responsive and consistent UI.
- **Application Layer (Backend):** FastAPI (Python) handles business logic, multi-factor recommendation, and AI integration.
- **Data Layer (Database):** PostgreSQL stores metadata, time-series price data, news sentiment, and portfolio information.
- **External Integrations:** OpenAI API for the educational chatbot, Alpha Vantage for news sentiment, FRED for macroeconomic indicators, and Yahoo Finance / ETF metadata sources for historical price and reference data.

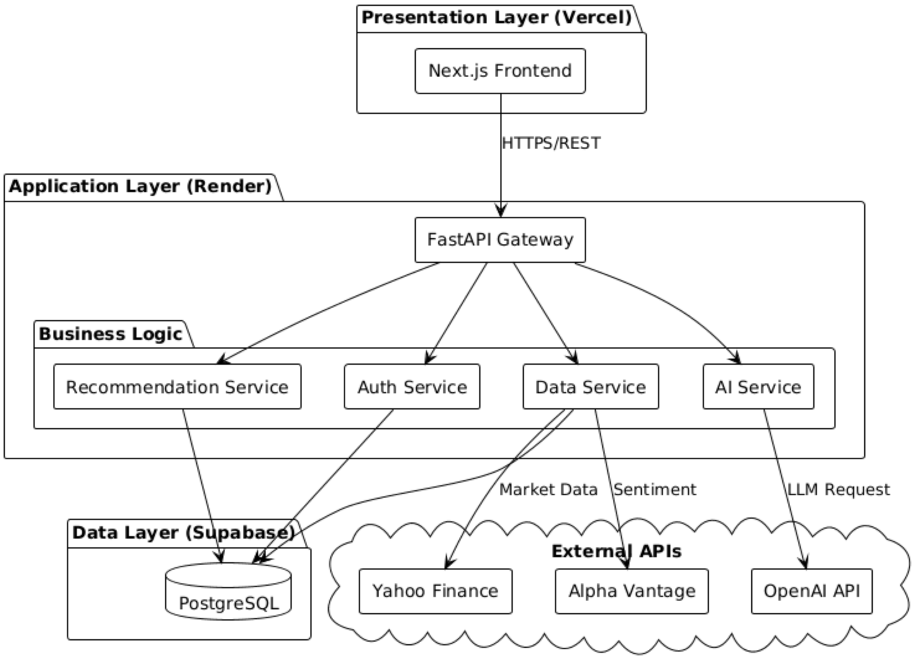

Figure 3.2: System Architecture

3.6 Sequence Diagrams

This section presents key interaction flows across frontend, backend APIs, service logic, and database operations. These sequence diagrams complement the static architecture and show the order of runtime messages for critical user journeys.

3.6.1 Login and Auth Session

The authentication sequence captures credential submission, backend validation, JWT issuance, token persistence in the frontend context, and session verification through the profile endpoint.

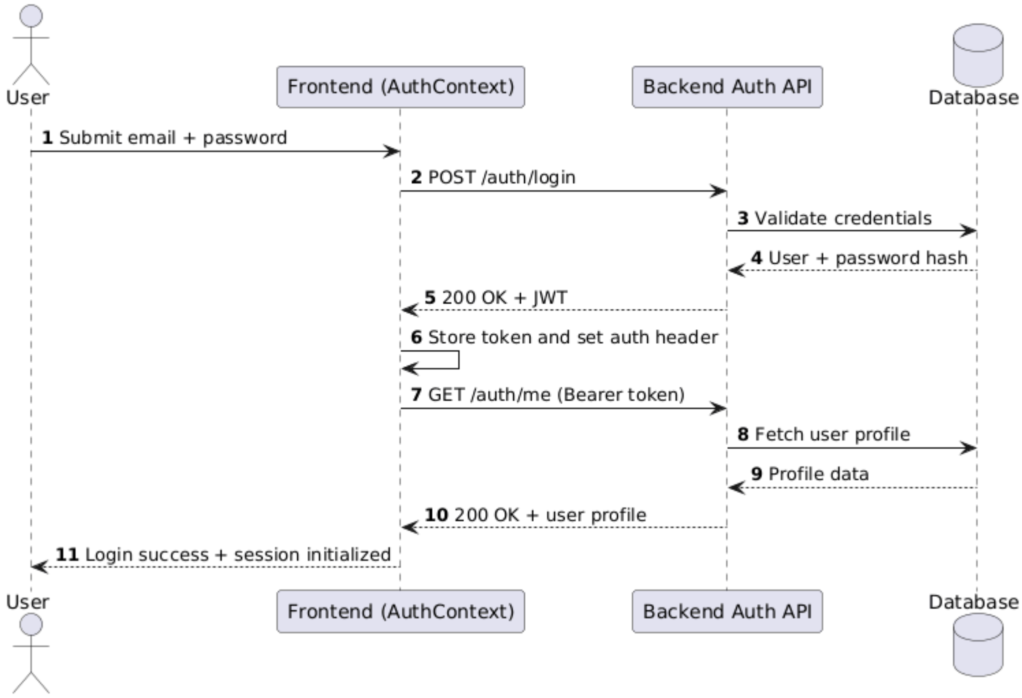

Figure 3.3: Sequence Diagram: Login and Auth Session

3.6.2 Risk Quiz to Recommendation

This sequence describes how quiz questions are loaded, responses are submitted, a risk profile is calculated, and recommendation results are retrieved and rendered.

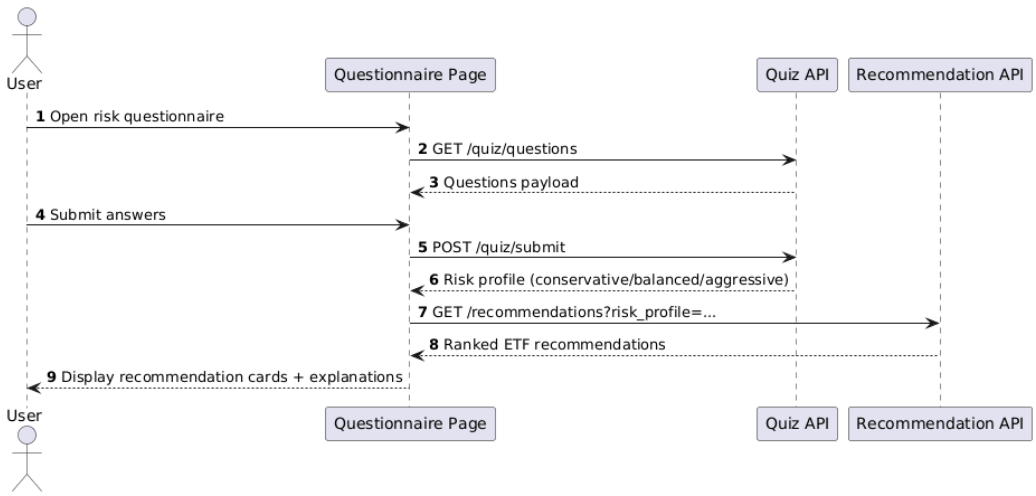

Figure 3.4: Sequence Diagram: Risk Quiz to Recommendation

3.6.3 Portfolio and Wallet Workflow

The portfolio-wallet interaction shows holding creation or updates, optional wallet assignment, price retrieval, recomputation of valuation and P/L, and grouped presentation.

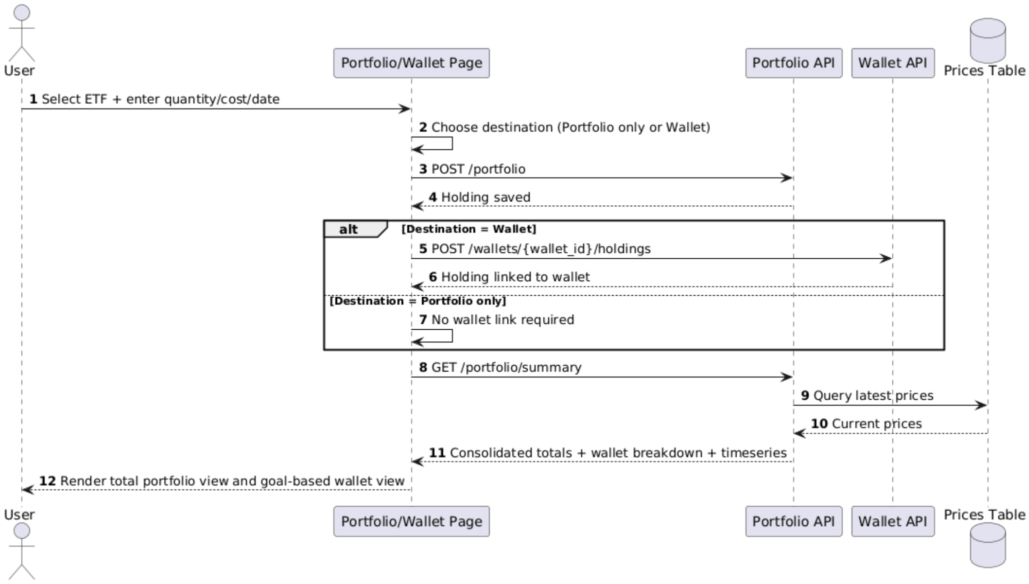

Figure 3.5: Sequence Diagram: Portfolio and Wallet Workflow

3.6.4 Scenario Analysis and VaR

This sequence outlines how users select scenario targets (portfolio, wallet, or ETF), run scenario impact analysis, and compute VaR/CVaR for downside-risk evaluation.

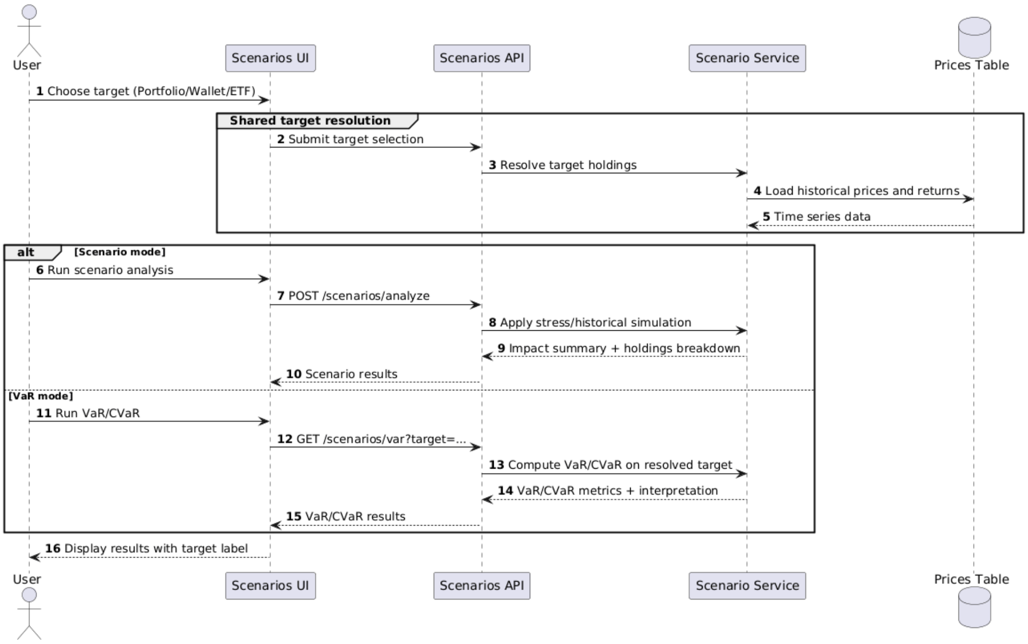

Figure 3.6: Sequence Diagram: Scenario Analysis and VaR

3.7 Database Design

The database schema ensures relational integrity, supports time-series financial data, and facilitates sentiment analysis. It is normalized for data consistency and complex queries across market dimensions:

- Users: The `users` table manages secure authentication and stores profile attributes including risk profile.
- ETFs (Master Data): The `etfs` table stores ETF metadata including expense ratios, AUM, dividends, and technical indicators.
- ETF Prices: The `etf_prices` table maintains historical OHLCV data linked to `etfs` for time-series analysis.
- Portfolios: Records user holdings with quantity, purchase price, and purchase date.
- Wallets: The `wallets`, `wallet_profiles`, and `wallet_holdings` tables support goal-based organization, wallet-level risk preferences, and wallet allocations.

- News and Sentiment: `news` stores articles and `news_tickers` maps them to tickers for sentiment scoring.
- Macro Indicators: Tracks key economic metrics like GDP, interest rates, and CPI for recommendations.

Key relationships include one-to-many links from users to portfolios and wallets, a one-to-one link between wallets and wallet profiles, and ticker-linked associations between wallet holdings, portfolio holdings, and ETF master data.

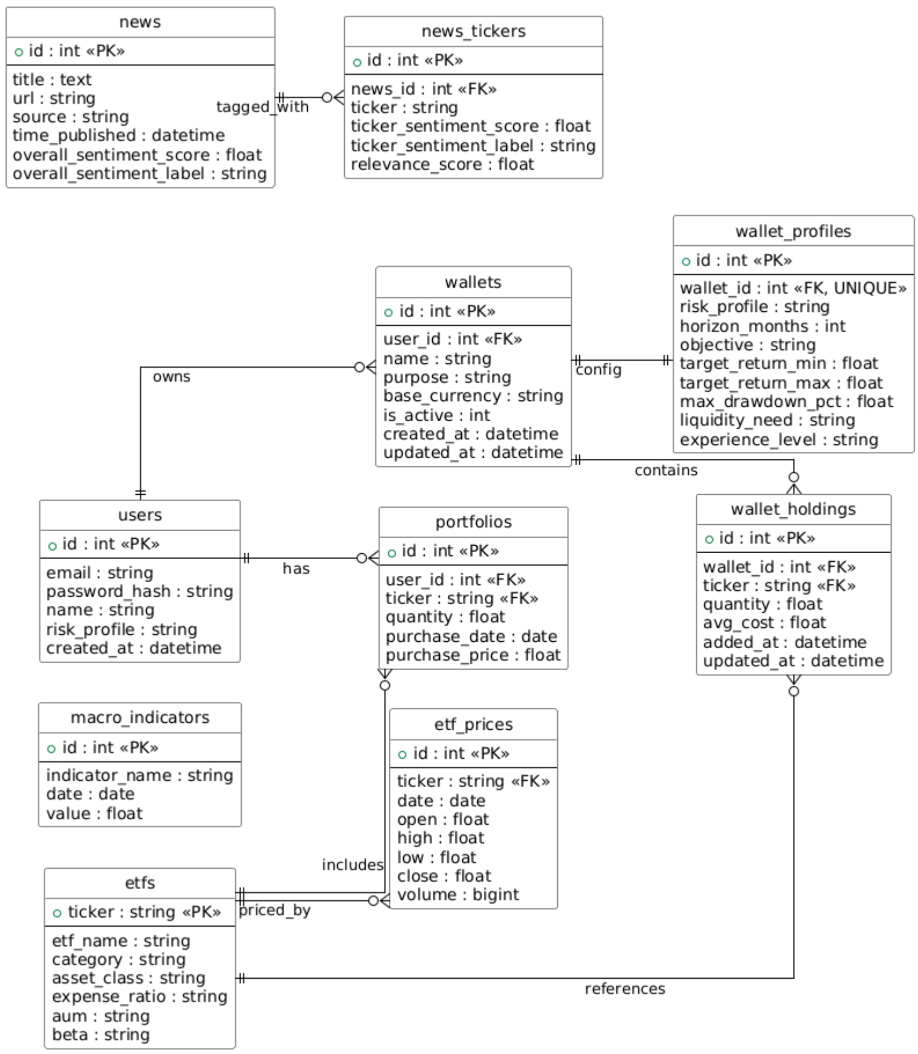

Figure 3.7: Entity-Relationship Diagram

3.8 API Design

The system is built as a RESTful API using FastAPI, following a stateless architecture and standard HTTP methods. All endpoints return data in JSON format and are secured via HTTPS. Protected routes require JWT authentication with a validity of seven days, and passwords are securely hashed using bcrypt. Public data endpoints (for example ETF discovery and recommendation retrieval by profile) are available without authentication, while user-specific operations (portfolio, wallets, profile updates, and chatbot requests) require a valid bearer token. Rate limiting can be enforced at the deployment gateway layer for operational stability.

Category	Method	Endpoint	Description	Auth
Auth	POST	`/auth/login`	Authenticate user	No
	POST	`/auth/signup`	Register new account	No
	GET	`/auth/me`	Get current user profile	Yes
	PUT	`/auth/me`	Update current user profile	Yes
ETFs	GET	`/etfs`	List and filter ETFs	No
	GET	`/etfs/{ticker}`	Get ETF metadata	No
	GET	`/etfs/{ticker}/prices`	Get historical prices	No
	GET	`/etfs/{ticker}/metrics`	Get ETF risk/performance metrics	No
Portfolio	GET	`/portfolio`	List user holdings	Yes
	POST	`/portfolio`	Add holding	Yes
	PUT	`/portfolio/{ticker}`	Update holding	Yes
	DELETE	`/portfolio/{ticker}`	Remove holding	Yes
Wallets	GET	`/wallets`	List user wallets	Yes
	POST	`/wallets`	Create wallet	Yes
	PUT	`/wallets/{wallet_id}/profile`	Upsert wallet profile	Yes
Analysis	GET	`/recommendations`	Risk-based recommendations by profile	No
	POST	`/scenarios/analyze`	Run stress tests	Yes
	GET	`/scenarios/var`	Calculate VaR/CVaR	Yes
	POST	`/chatbot`	AI assistant query	Yes
Market	GET	`/macro/indicators`	Economic indicators	No
News	GET	`/news/ticker/{ticker}`	News and sentiment for ETF	No
	GET	`/news/portfolio/{user_id}`	News for user's portfolio	Yes
	GET	`/news/alerts/{user_id}`	News alerts for user	Yes

Continued on next page

Table 3.2: Summary of Core API Endpoints

3.9 User Interface Design

The platform includes the following key user interfaces. Core screens are illustrated in the figures below, while additional implemented pages (wallet management, scenario analysis, and profile settings) extend the operational workflow.

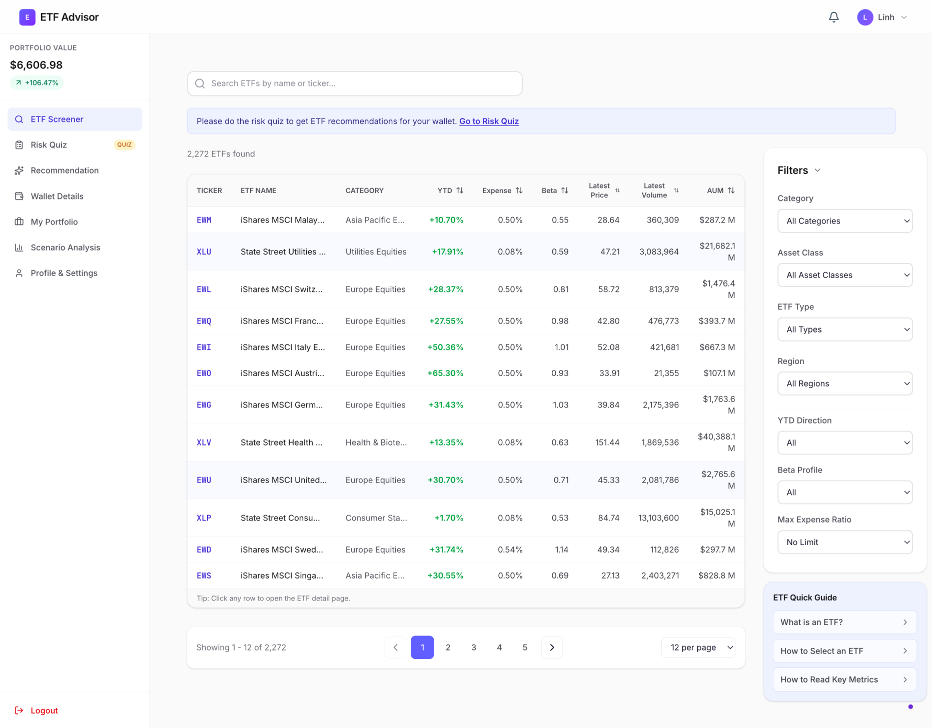

Figure 3.8: ETF Discovery UI

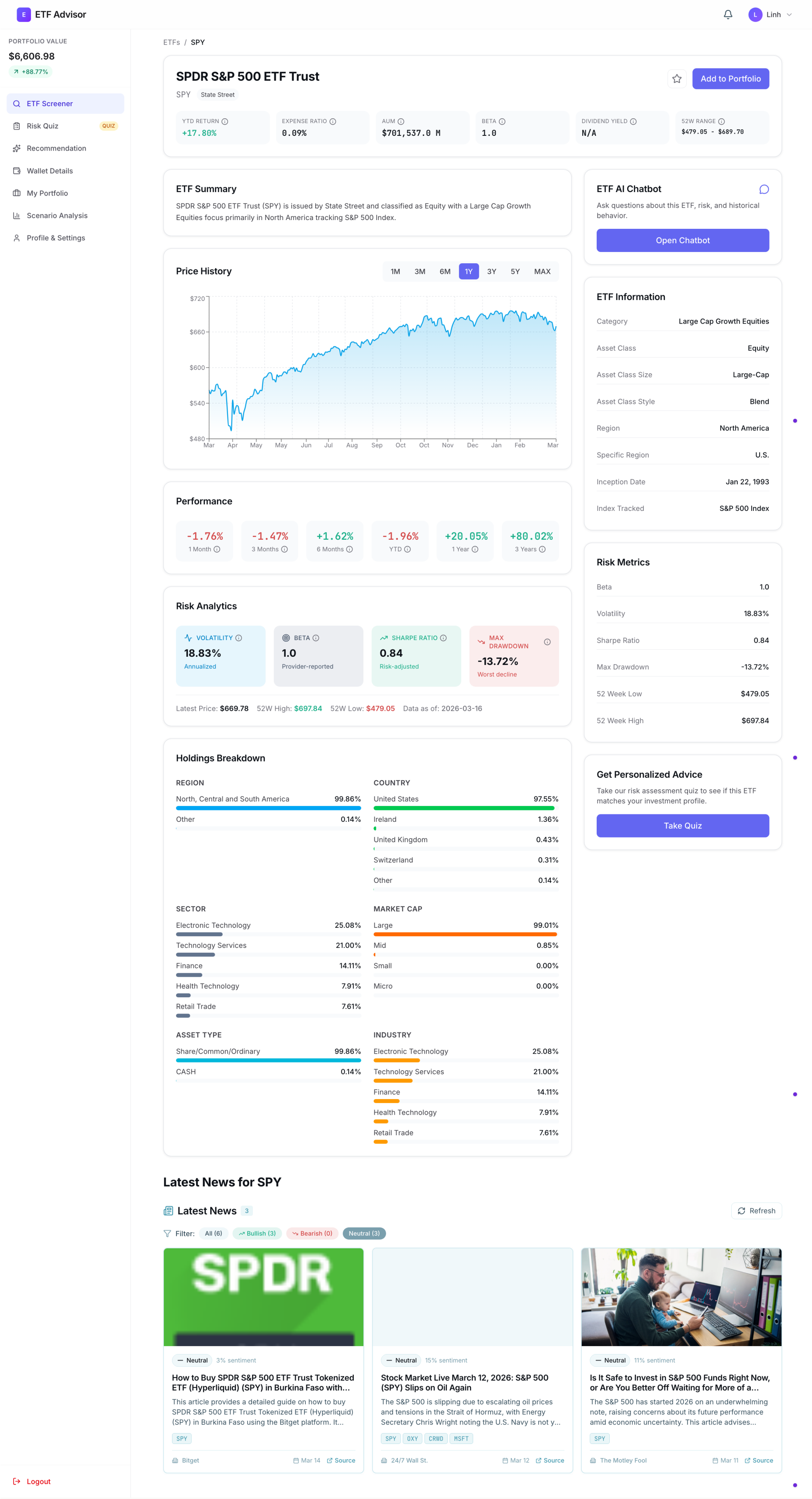

Figure 3.9: ETF Details with AI Chatbot

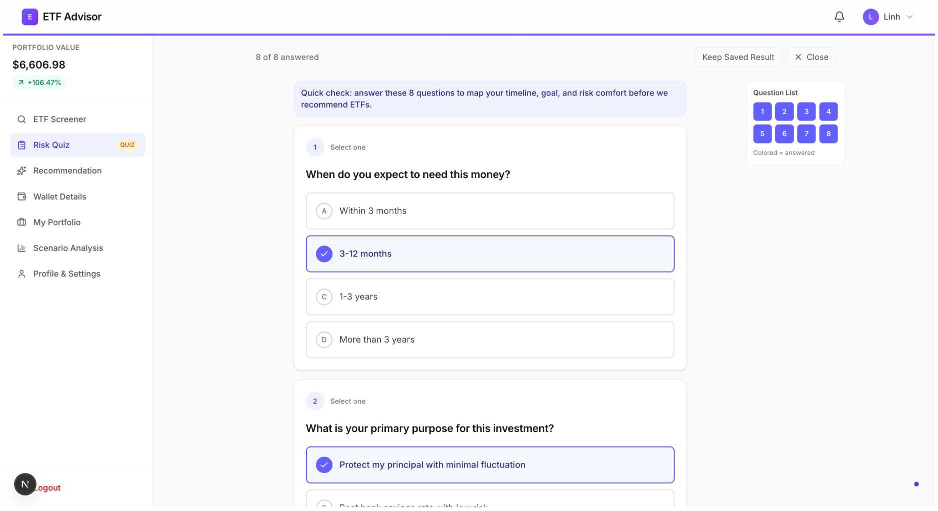

Figure 3.10: Risk Assessment Questionnaire UI

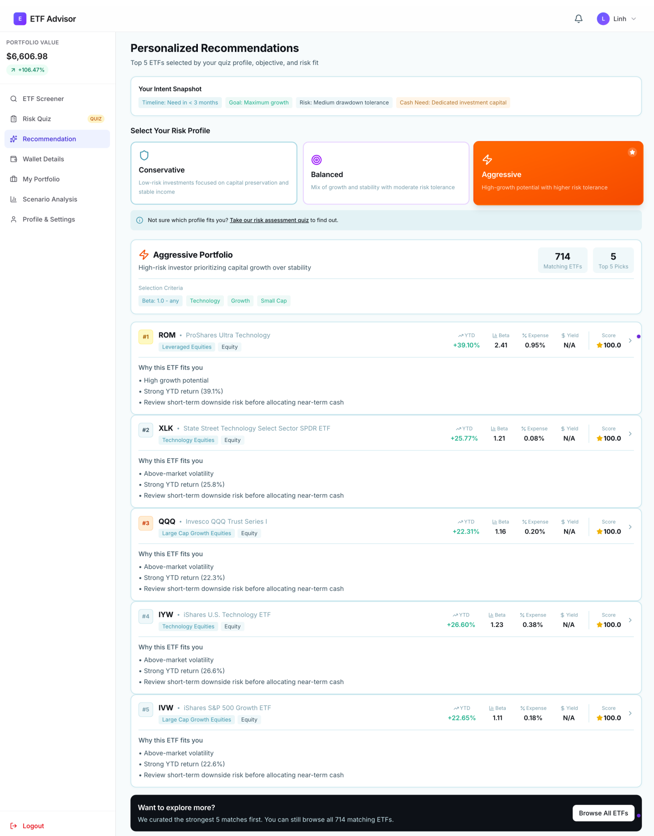

Figure 3.11: ETF Recommendation Result UI

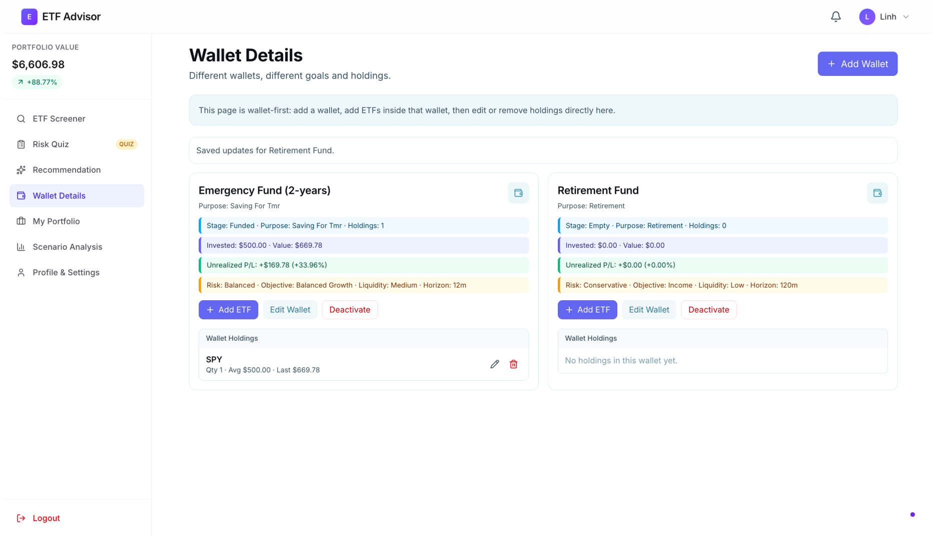

Figure 3.12: Wallet Details Page UI

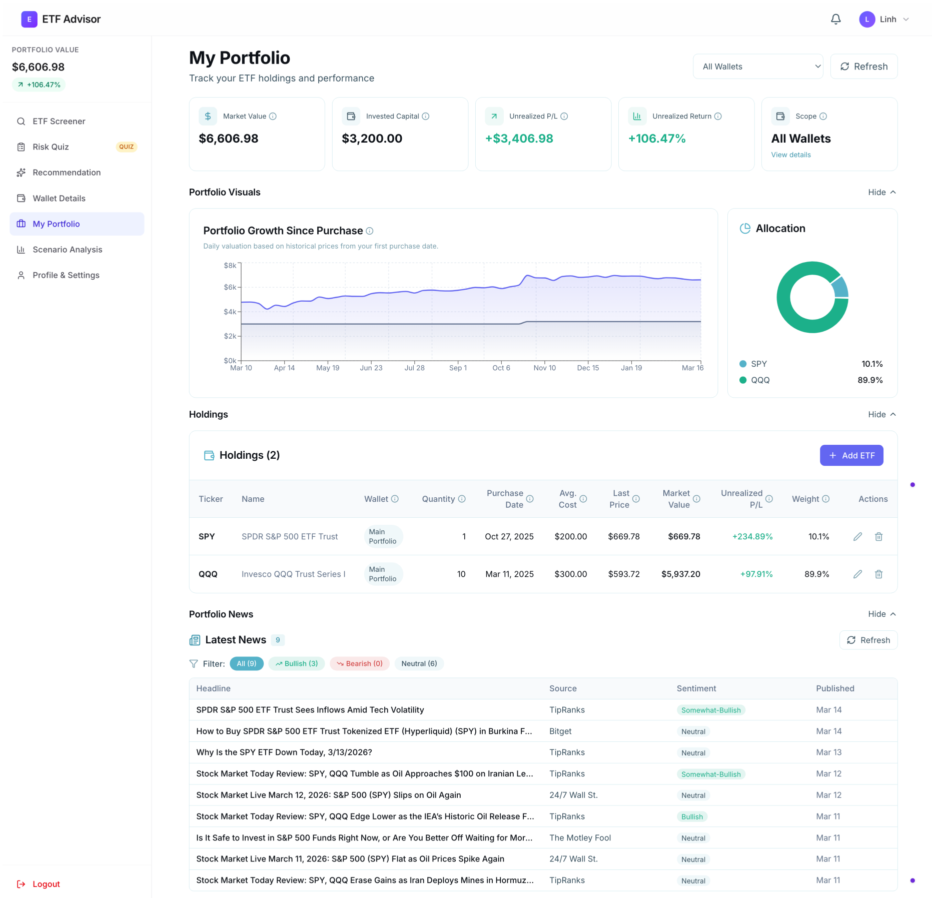

Figure 3.13: User Portfolio Dashboard UI

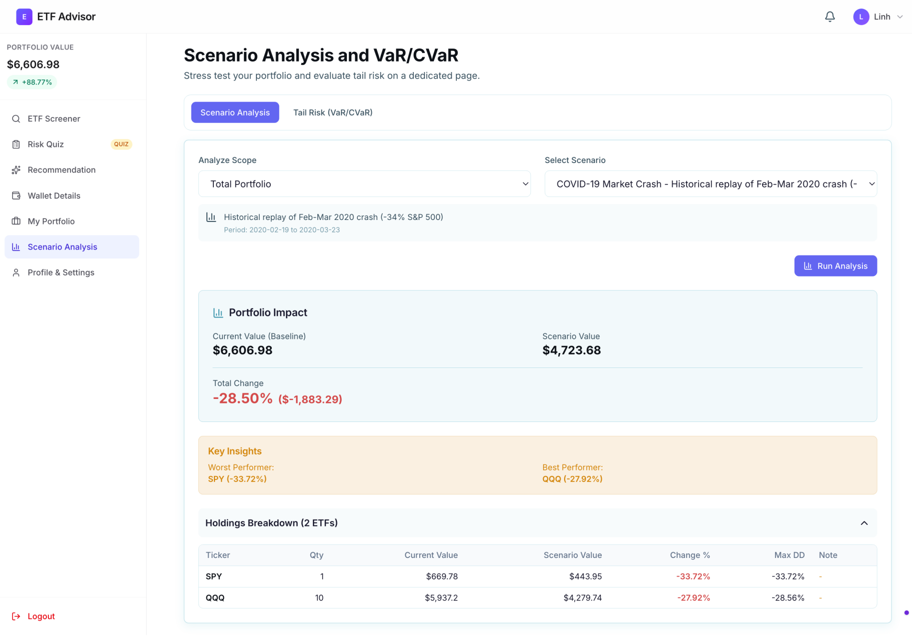

Figure 3.14: Scenario Analysis and VaR UI

Chapter 4

System Implementation

4.1 Technology Stack Overview

The system adopts a modern full-stack architecture designed for scalability, type safety, and efficient handling of financial data. Each component in the stack was selected based on its maturity, ecosystem support, and suitability for real-time analytical workloads. To improve traceability, the versions listed in this section are aligned with the current project configuration files, which are frontend/package.json, backend/requirements.txt, and backend/runtime.txt at the time of writing.

4.1.1 Frontend Technologies

Technology	Version	Purpose
Next.js	14.2.5	React framework with App Router for SSR/SSG
TypeScript	5.x	Static typing and improved maintainability
Tailwind CSS	4.x	Utility-first responsive UI styling
Recharts	3.6.0	Financial data visualization
TanStack Query	5.90.x	Server-state management and caching
Axios	1.13.x	HTTP client for API communication

Table 4.1: Frontend Technology Stack

4.1.2 Backend Technologies

Technology	Version	Purpose
Python	3.11.7	Core backend language
FastAPI	$\geq$ 0.104.0	Asynchronous API framework
SQLAlchemy	$\geq$ 2.0.23	ORM for database abstraction
Pydantic	$\geq$ 2.5.0	Data validation and serialization
Pandas	$\geq$ 1.5.0	Financial and time-series computation
APScheduler	$\geq$ 3.10.4	Scheduled background tasks (news refresh jobs)
bcrypt	$\geq$ 4.0.1	Secure password hashing
python-jose	$\geq$ 3.3.0	JWT token handling

Table 4.2: Backend Technology Stack

4.1.3 Database and External Services

Service	Provider	Purpose
PostgreSQL	Supabase	Relational data storage (5.7M+ records)
OpenAI API	OpenAI	LLM-powered educational chatbot responses
Alpha Vantage	Alpha Vantage	News sentiment analysis
FRED	Federal Reserve Economic Data	Macroeconomic indicators
Yahoo Finance	yfinance	Historical OHLCV price data

Table 4.3: Database and External Services

4.1.4 Deployment Infrastructure

Component	Platform	Configuration
Frontend	Vercel	Edge CDN, GitHub CI/CD
Backend	Render	Docker, Gunicorn + Uvicorn
Database	Supabase	AWS ap-southeast-1, pooling enabled

Table 4.4: Deployment Infrastructure

4.2 Data Pipeline Implementation

The data pipeline forms the backbone of the platform and is responsible for aggregating, cleaning, validating, and storing financial data across more than 2,200 ETFs.

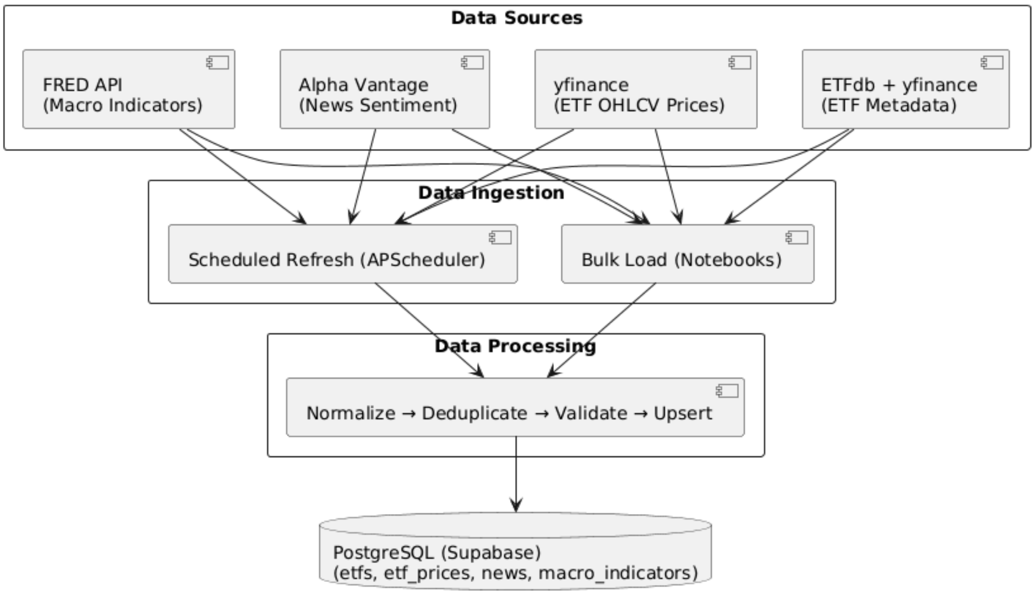

Figure 4.1: Data Pipeline Flowchart

4.2.1 Data Collection

ETF metadata, historical prices, and news sentiment are collected from multiple authoritative sources to ensure data completeness and accuracy.

ETF Metadata Collection: ETF metadata is collected using automated scripts that scrape ETFdb for U.S.-listed ETFs and utilize Yahoo Finance for Singapore-listed ETFs. All fields are normalized into a unified schema, ensuring consistency across markets.

Historical Price Collection: The initial 15-year OHLCV dataset was collected via Jupyter notebooks using the yfinance library, exported to CSV files, and bulk-loaded into the database. Ongoing incremental updates are handled by app/services/price_service.py, which is invoked automatically by the APScheduler job at application startup and by a lightweight CLI script for manual or backfill runs.

```
def refresh_all_prices(db, tickers=None, default_lookback_days=7):
    """Incrementally refresh OHLCV via yfinance (batched, skip up-to
        -date)."""
    all_tickers = tickers or _get_all_tickers(db)
    last_dates = _get_last_price_dates(db, all_tickers)

    today = datetime.now().date()
    needs_update = [
        (t, (last_dates[t] + timedelta(days=1)) if t in last_dates
         else (today - timedelta(days=default_lookback_days)))
        for t in all_tickers
        if (t not in last_dates or last_dates[t] + timedelta(days=1)
            < today)
    ]

    if not needs_update: return
    min_start = min(s for _, s in needs_update)

    for i in range(0, len(needs_update), 50):
        batch = needs_update[i:i+50]
        raw = yf.download(" ".join(t for t,_ in batch),
                          start=min_start, end=today,
                          group_by="ticker", auto_adjust=True,
                              progress=False)

        parsed = _parse_download(raw, [t for t,_ in batch])
        for t, rows in parsed.items():
            _upsert_batch(db, t, rows, existing_set)
```

Listing 4.1: Batch OHLCV Refresh via yfinance (price_service.py)

News Data Collection: Alpha Vantage's `NEWS_SENTIMENT` endpoint is queried with application-level throttling. The implemented limiter enforces both per-minute and per-day caps to prevent quota violations, and parsed sentiment/relevance fields are persisted for downstream analytics.

4.2.2 Automated Refresh Strategy

While the initial bulk load covers historical data, several datasets require periodic refresh to remain accurate. Four APScheduler jobs are designed to handle this:

- **OHLCV prices** (weekdays 20:00 ET): price_service.refresh_all_prices() fetches only the missing date window per ticker via yfinance batch download (50 tickers/call), covering all 2,200+ ETFs. A CLI wrapper supports ad-hoc backfill runs.
- **News sentiment** (daily 18:00 ET): news_service.refresh_news_for_tickers() batches the top 100 ETFs by AUM into groups of 5 and calls Alpha Vantage `NEWS_SENTIMENT`, capped at 25 calls/day and 5 calls/min.
- **ETF metadata** (monthly): A scheduled job re-queries ETFdb and `yfinance` to capture newly listed ETFs and updates to AUM, expense ratios, and category classifications, ensuring the ETF catalogue stays current.
- **Macro indicators** (monthly): A scheduled job pulls the latest FRED series (e.g. CPI, GDP, Federal Funds Rate) to incorporate newly released data points as economic indicators are published.

4.2.3 Data Processing and Validation

Before storage, raw data undergoes multiple preprocessing stages:

- Normalization: Conversion of percentage and currency strings to numeric formats.
- Gap Filling: Forward-filling missing dates for continuous time series.
- Deduplication: Enforcement of composite keys on (ticker, date).

- Validation: Sanity checks on prices, dates, and ticker formats.

4.2.4 Database Population

Bulk data ingestion is optimized using PostgreSQL's COPY command.

```
psql $DATABASE_URL -c "\COPY etf_prices FROM 'prices.csv' WITH CSV HEADER"
```

Listing 4.2: Bulk Data Upload Script

4.3 Backend Implementation

The backend is implemented using FastAPI and follows a service-oriented, modular architecture.

4.3.1 Project Structure

File / Folder	**Description**
backend/	Root folder
app/	Application package (FastAPI)
main.py	App entry, scheduler, route registration
auth.py	JWT, password hashing, auth dependencies
database.py	SQLAlchemy engine, SessionLocal, Base, get_db
models.py	ORM models (etfs, etf_prices, users, wallets, news)
schemas.py	Pydantic request/response schemas
routes/	APIRouter modules (auth, etfs, prices, news, ...)
services/	Business logic and background helpers
scripts/	CLI utilities (e.g., update_prices_simple.py)
requirements.txt	Python dependencies

Table 4.5: Backend Project Structure

The backend exposes a RESTful API organized into logical modules as defined in the system design (see Chapter 3, Table 3.2). The implementation uses 'APIRouter' to modularize the codebase, with specific routers for authentication, ETF data, portfolio management, and analytical services.

4.3.2 Database Design

The schema of the database is optimised for financial time-series data and rapid retrieval. Key tables include etf_prices, which stores millions of records, partitioned and indexed for performance.

```sql
CREATE TABLE etf_prices (
    ticker VARCHAR(10) NOT NULL,
    date DATE NOT NULL,
    open DECIMAL(10, 4),
    high DECIMAL(10, 4),
    low DECIMAL(10, 4),
    close DECIMAL(10, 4),
    volume BIGINT,
    PRIMARY KEY (ticker, date)
);
CREATE INDEX idx_prices_ticker_date ON etf_prices (ticker, date DESC
    );

CREATE TABLE news_tickers (
    news_id INTEGER REFERENCES news(id),
    ticker VARCHAR(10) REFERENCES etfs(ticker),
    sentiment_score FLOAT,
    relevance_score FLOAT,
    PRIMARY KEY (news_id, ticker)
);
```

Listing 4.3: Core Tables Schema

Other relational tables include etfs (ETF metadata and attributes such as AUM, expense ratio, category, and a snapshot of holdings), news and news_tickers (news articles and their ticker-level sentiment/relevance links), macro_indicators (FRED series values with timestamps), and user-related tables (users, portfolios, wallets, wallet_holdings) which record user profiles, portfolio compositions, wallet goals, and holdings history. These tables are normalised where appropriate; referential integrity is enforced with foreign-key constraints (e.g., news_tickers.news_id -> news.id and wallet_holdings.portfolio_id -> portfolios.id). Indexes are created on frequently queried columns (for example, ticker, date, user_id, portfolio_id) to accelerate joins and lookups.

4.3.3 Metrics Calculation

Financial metrics are calculated on-demand using vectorized Pandas operations to ensure freshness and accuracy. These are some examples of functions that calculate financial metrics, not being collected via APIs, such as sharpe ratio and maximum drawdown.

Sharpe Ratio: Measures risk-adjusted return using the 10-year Treasury yield as the risk-free rate.

```python
def calculate_sharpe(returns: pd.Series, risk_free_rate: float =
    0.04) -> float:
    annualized_return = returns.mean() * 252
    annualized_vol = returns.std() * math.sqrt(252)
    return (annualized_return - risk_free_rate) / annualized_vol
```

Listing 4.4: Sharpe Ratio Calculation

Maximum Drawdown: Computes the largest peak-to-trough decline in the price history.

```python
def calculate_max_drawdown(prices: pd.Series) -> float:
    running_max = prices.cummax()
    drawdown = (prices - running_max) / running_max
    return drawdown.min() * 100  # Return as percentage
```

Listing 4.5: Maximum Drawdown Calculation

4.3.4 Risk Profiling

User risk tolerance is derived from a questionnaire-driven scoring workflow implemented through backend quiz endpoints and frontend questionnaire pages. The current quiz contains 8 multiple-choice questions, with per-option points defined in "data/user_info/quiz_qus.json". Scores are summed and mapped into three operational profiles used by downstream recommendation logic: **conservative** (0–260), **balanced** (261–500), and **aggressive** (501–720). Because thresholds and point weights are loaded from configuration at runtime, profile boundaries can be adjusted without changing API contracts. When a quiz is submitted by an authenticated user, the latest profile is persisted to "users.risk_profile" for reuse across later sessions.

Profile	**Practical Interpretation**
Conservative	Capital preservation and lower volatility preference
Balanced	Mixed growth and stability preference
Aggressive	Higher return target with higher risk tolerance

Table 4.6: Implemented Risk Profile Categories

More details about risk questions in Appendix B.

4.3.5 Recommendation Engine

The current recommendation engine is implemented as a **rule-based multi-factor scorer** with profile-aware filtering in "routes/recommendations.py". It first removes ETFs in profile-specific avoided categories, then applies profile-specific beta hard filters where defined (for example conservative: beta ≤ 0.8, aggressive: beta ≥ 1.0). Balanced recommendations are primarily shaped by scoring rather than strict beta exclusion.

Each candidate starts from a baseline score of 50 and receives additive bonuses or penalties based on parsed ETF attributes. The API returns ETFs with score ≥ 40, sorted descending by score, and also supports optional category and region filters at query time.

The following listing is a simplified representation of the implemented scoring logic in the backend.

Implemented scoring signals include:

- beta alignment with profile expectations,
- return and dividend heuristics (profile-dependent weighting),
- low expense ratio bonus (e.g., expense ratio < 0.2%),
- category and asset-class matching bonuses,
- risk-category penalties for avoided categories.

```python
def calculate_recommendation_score(etf, profile):
    score = 50
    reasons = []

    beta = parse_beta(etf.beta)
    ytd_return = parse_percentage(etf.return_ytd)
    dividend = parse_percentage(etf.annual_dividend_yield)
    fee = parse_percentage(etf.expense_ratio)

    # Beta adjustments by profile
    if profile == "conservative":
        if beta is not None and beta < 0.5:
            score += 20; reasons.append("Very low volatility")
        elif beta is not None and beta < 0.8:
            score += 15; reasons.append("Low volatility")
    elif profile == "balanced":
        if beta is not None and 0.9 <= beta <= 1.1:
            score += 15; reasons.append("Market-aligned
                volatility")
    elif profile == "aggressive":
        if beta is not None and beta > 1.3:
            score += 20; reasons.append("High growth potential
                ")
        elif beta is not None and beta > 1.1:
```

```python
            score += 10; reasons.append("Above-market volatility")

    # Return/dividend heuristics
    if profile == "conservative" and dividend is not None and dividend > 3:
        score += 15
    if profile == "balanced" and dividend is not None and dividend > 2:
        score += 10
    if profile == "aggressive" and ytd_return is not None and ytd_return > 20:
        score += 20
    if profile == "balanced" and ytd_return is not None and ytd_return > 10:
        score += 10

    # Cost/category/class effects
    if fee is not None and fee < 0.2:
        score += 10
    if matches_preferred_category(etf, profile):
        score += 15
    if matches_preferred_asset_class(etf, profile):
        score += 10
    if in_avoid_category(etf, profile):
        score -= 30

    score = min(100, max(0, score))
    return score, reasons[:3]
```

Listing 4.6: Rule-Based Recommendation Scoring (Simplified), basicstyle=

4.3.6 Scenario Analysis and VaR/CVaR

The scenario module supports historical replay and stress tests via /scenarios/analyze at the portfolio, wallet, or single-ETF level. Implemented scenarios include a historical window (COVID-19 crash, 2020-02-19 to 2020-03-23) and uniform shocks (-10%, -20%, -30%).

For each holding, the backend computes a base valuation (latest close or purchase/average cost) and reports scenario value, change (amount%), and portfolio-level aggregate impact. The COVID replay also reports per-holding max drawdown.

The VaR endpoint (/scenarios/var) calculates VaR and CVaR using historical (empirical tail) and parametric (normal approximation) methods. Supported confidence levels are 0.90, 0.95, 0.99, with a configurable lookback window (30–1000 days).

```python
def calculate_var(db, holdings, confidence_level=0.95, time_horizon_days=252):
    holding_values = {t: qty * latest_or_purchase_price(t) for t, qty in holdings}
    portfolio_value = sum(holding_values.values())
    portfolio_returns = aggregate_weighted_returns(holding_values, time_horizon_days)
    sorted_returns = sorted(portfolio_returns)

    var_cut = percentile(sorted_returns, 1 - confidence_level)
    hist_var_amt = abs(var_cut) * portfolio_value
    hist_tail = [r for r in sorted_returns if r <= var_cut]
    hist_cvar_amt = abs(mean(hist_tail)) * portfolio_value

    mu, sigma = mean(portfolio_returns), std(portfolio_returns)
    z = {0.90: 1.28, 0.95: 1.65, 0.99: 2.33}[confidence_level]
    para_var_amt = abs(mu - z * sigma) * portfolio_value
    para_cvar_amt = abs(mu - sigma * phi(z) / (1 - confidence_level)) * portfolio_value

    return build_response(hist_var_amt, para_var_amt, hist_cvar_amt, para_cvar_amt)
```

Listing 4.7: Implemented Portfolio VaR/CVaR Flow (Simplified)

4.3.7 AI Chatbot Service

The chatbot integrates OpenAI LLM APIs for educational Q&A. To reduce hallucinations and keep responses focused, the service follows retrieval-style grounding by injecting current ETF metadata and computed metrics into the prompt context before model invocation.

```
def construct_prompt(etf_data, user_question):
    context = f"""
    You are a financial assistant analyzing {etf_data['name']} ({etf_data['ticker']}).
    Current Price: ${etf_data['price']} | Expense Ratio: {etf_data['expense_ratio']}%
    YTD Return: {etf_data['ytd_return']}% | Beta: {etf_data['beta']}
    Top Holdings: {', '.join(etf_data['top_holdings'])}
    """

    return [
        {"role": "system", "content": context + "\nProvide factual educational info. Do not give financial advice."},
        {"role": "user", "content": user_question}
    ]
```

Listing 4.8: AI System Prompt Construction

4.3.8 Authentication and Security

Security is enforced using JWT (JSON Web Tokens) for stateless authentication and bcrypt for password hashing. This ensures that sensitive user credentials are never stored in plain text and that API routes are protected against unauthorized access.

```
pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

def get_password_hash(password: str) -> str:
    return pwd_context.hash(password)

def verify_password(plain_password: str, hashed_password: str) -> bool:
```

```
    return pwd_context.verify(plain_password, hashed_password)
```

Listing 4.9: Password Hashing and Verification

4.4 Frontend Implementation

The frontend is developed using Next.js 14 with a component-driven architecture and responsive design principles.

File / Folder	Description
frontend/	Root folder
src/	Source code folder
app/	Next.js App Router pages
page.tsx	Landing page (/)
layout.tsx	Root layout with providers
login/page.tsx	Login page
signup/page.tsx	Signup page
questionnaire/page.tsx	User questionnaire page
recommendations/page.tsx	Recommendations page
etfs/	ETF discovery pages
page.tsx	ETF discovery main page
[ticker]/page.tsx	Individual ETF details page
portfolio/page.tsx	Portfolio overview page
wallets/page.tsx	Wallets overview page
scenarios/page.tsx	Scenario analysis page
profile/page.tsx	User profile page
components/	Reusable UI components
AppShell.tsx	Main app shell layout
Header.tsx	Header component
NewsFeed.tsx	News feed component

Continued on next page

Table 4.7 – *Continued from previous page*

File / Folder	Description
ScenarioAnalysis.tsx	Scenario analysis component
VarAnalysis.tsx	VaR/CVaR analysis component
contexts/	React context providers
AuthContext.tsx	Authentication context
lib/	Utility and helper libraries
api.ts	Axios API client
providers.tsx	Query/Auth providers
package.json	NPM dependencies

Table 4.8: Frontend Project Structure

4.4.1 Frontend Routes

Page	Route	Key Features
Landing	/	Feature overview and call-to-action
Login	/login	Authentication
Questionnaire	/questionnaire	Risk assessment wizard
Recommendations	/recommendations	Profile-aligned ETF ranking and rationale
ETF Discovery	/etfs	Search and filtering
ETF Details	/etfs/[ticker]	Charts, metrics, chatbot
Portfolio	/portfolio	Holdings, valuation, and news
Wallets	/wallets	Goal-based wallet management
Scenario Analysis	/scenarios	Stress testing and VaR/CVaR
Profile Settings	/profile	User profile and risk preference updates

Table 4.9: Frontend Route Map

4.5 Implementation Challenges

Developing a data-intensive financial platform presented several significant technical challenges. This section details the key issues encountered and the solutions implemented.

4.5.1 API Rate Limiting and Data Freshness

Challenge: The Alpha Vantage API (free tier) restricts usage to 5 calls per minute and 500 per day. Updating news for 2,000+ ETFs daily was impossible under these constraints.

Solution: I implemented a prioritization queue using `APScheduler`. The system updates the top 100 ETFs by AUM daily (requiring 100 calls spread over 20 minutes) and updates other ETFs on-demand when a user views their details page. I also utilized caching strategies to prevent redundant API calls for the same ticker within a 24-hour window.

4.5.2 Data Consistency and Gap Filling

Challenge: Financial data from newly listed ETFs often contains null values or non-contiguous dates, breaking chart rendering and metric calculations (e.g., volatility requires continuous returns).

Solution: A robust preprocessing pipeline was built using Pandas. Missing prices on weekends/holidays are handled via forward-filling (`ffill()`), ensuring no look-ahead bias. For metrics requiring strict window sizes (e.g., 252-day volatility), ETFs with insufficient history are flagged and excluded from certain risk analytics to prevent skewed results.

4.6 Deployment Configuration

The platform is deployed using a cloud-native architecture with clearly separated frontend, backend, and database services to ensure scalability, reliability, and ease of maintenance.

- Backend: FastAPI is containerized with Docker and deployed on Render using Gunicorn and Uvicorn workers.
- Frontend: The Next.js application is deployed on Vercel with automatic CI/CD from GitHub.
- Database: PostgreSQL on Supabase is configured with connection pooling, indexing, row-level security, and automated backups.

Chapter 5

Demonstration and Testing

This chapter first demonstrates the completed platform through its core user workflows, then presents the testing and evaluation results that validate system correctness, performance, and usability.

5.1 System Demonstration

This section walks through the platform's key features following the natural user journey, from onboarding to advanced risk analysis.

5.1.1 Registration

New users register with an optional display name, email, and password. A confirm-password field with a real-time match indicator validates input. Upon successful registration, the user is redirected to the risk questionnaire to begin personalisation.

5.1.2 Risk Profiling

The risk questionnaire comprises eight multiple-choice questions covering loss tolerance, time horizon, return expectations, reaction to downturns, income stability, liquidity needs, and portfolio goals. A progress bar tracks completion, and the "Get Results" button activates only once all questions are answered.

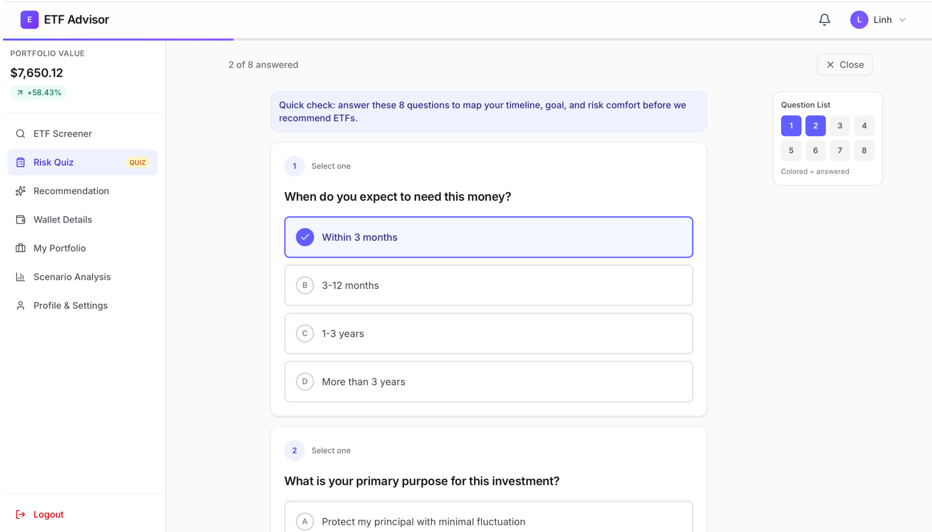

Figure 5.1: Risk profiling questionnaire with progress tracking

The system computes a weighted score and classifies the user as *Conservative*, *Balanced*, or *Aggressive*. The result screen shows the profile badge, total score, an intent snapshot (timeline, goal, drawdown comfort, cash priority), and a per-question breakdown. The user can then view their top ETF picks or edit their answers.

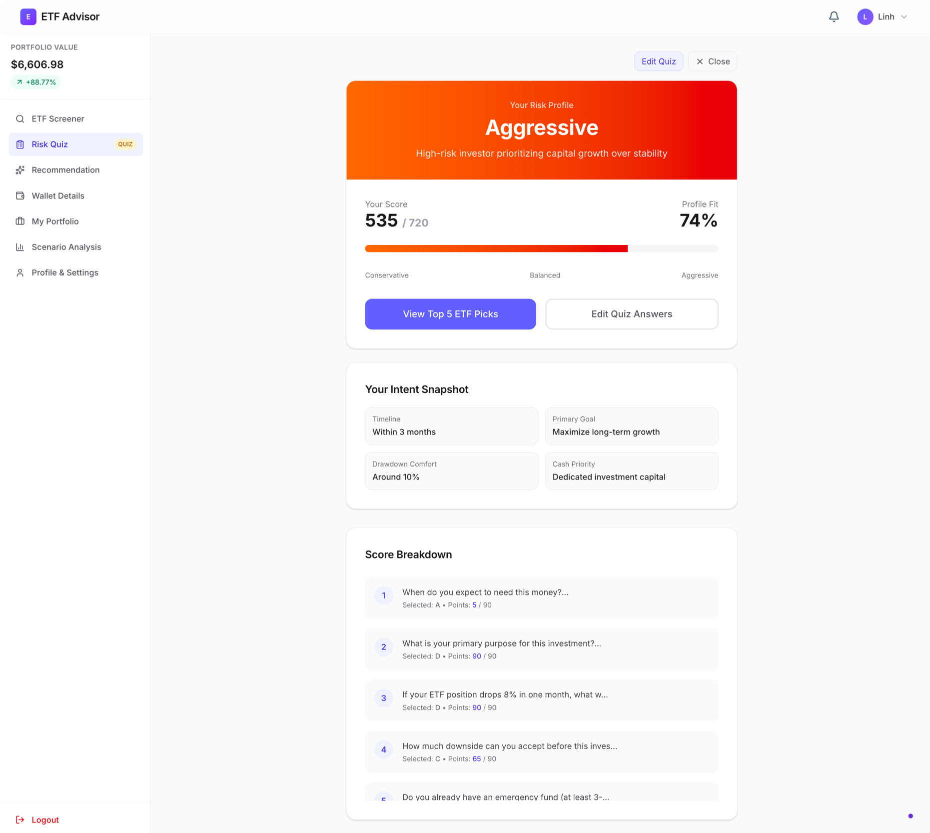

Figure 5.2: Risk quiz result with profile classification and intent snapshot

5.1.3 Personalised ETF Recommendations

The recommendations page displays five ETF cards ranked by the multi-factor score, pre-filtered to the user's risk profile. Each card shows the ticker, name, category, key metrics (YTD return, beta, expense ratio, dividend yield), a score out of 100, and reason labels explaining the match (e.g., "Low volatility," "Strong dividend yield"). Users can switch profiles to compare recommendations, click a card to view its detail page, or navigate to the full ETF discovery screen.

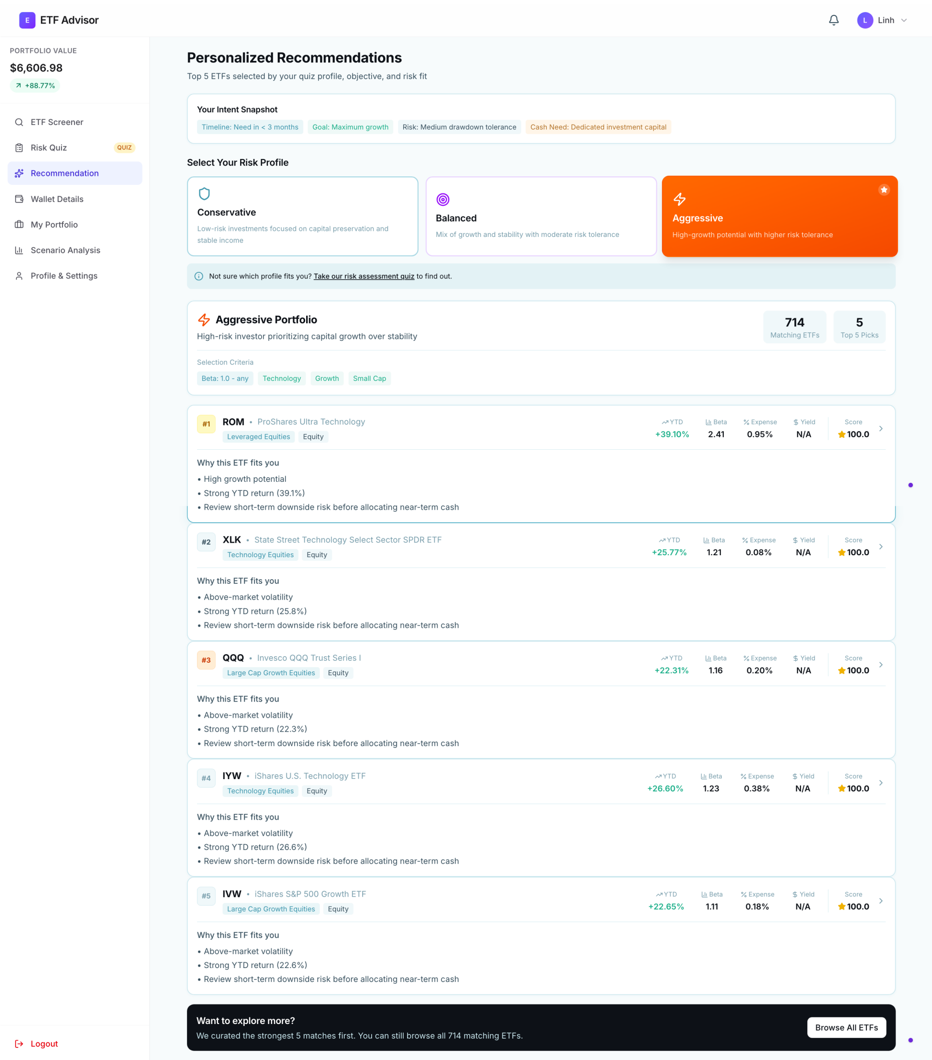

Figure 5.3: Personalised ETF recommendations with scores and explanatory labels

5.1.4 ETF Screener

The ETF Discovery tab provides access to all 2,272 ETFs via a search bar and filters for category, asset class, ETF type, region, YTD direction, beta profile, and expense ratio. Results appear in a sortable, paginated table. A collapsible "ETF Quick Guide" explains key concepts for beginner users.

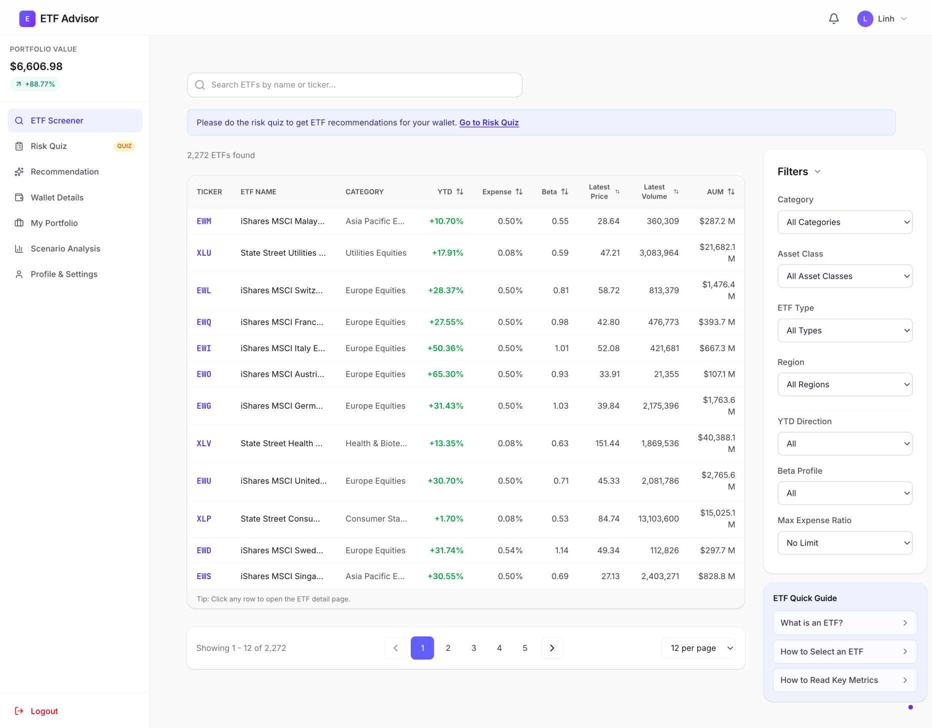

Figure 5.4: ETF Discovery page with search, filters, and sortable table

5.1.5 ETF Detail View

Selecting an ETF opens its detail page, which includes a quick-stats bar (YTD return, expense ratio, AUM, beta, dividend yield, 52-week range), an interactive price chart (1M–MAX), computed risk metrics (volatility, beta, Sharpe ratio, max drawdown), holdings breakdown, fund metadata, and a ticker-level news feed with sentiment scores.

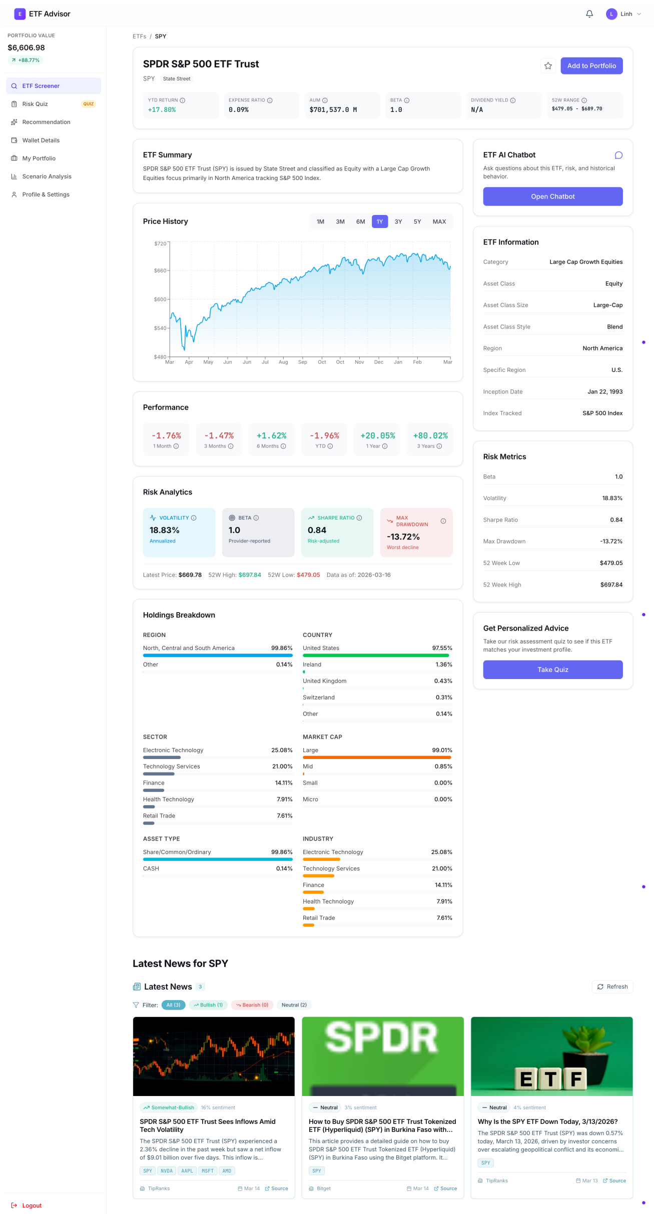

Figure 5.5: ETF detail page with price chart, risk metrics, and news feed

5.1.6 AI Chatbot Interaction

Each detail page includes an AI chatbot accessible via a floating button. The chatbot is grounded with the ETF's live metadata and metrics, enabling factual responses to questions such as "Is this ETF suitable for a conservative investor?" Every reply includes a financial disclaimer. Figure 5.6 shows a sample conversation.

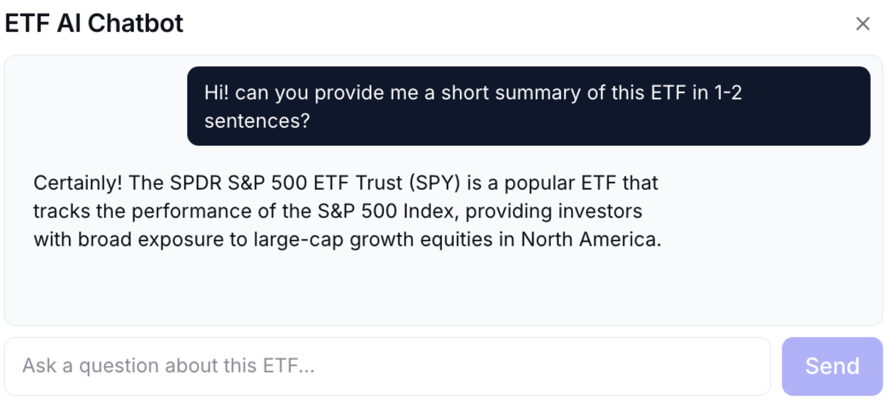

Figure 5.6: AI chatbot conversation grounded with live ETF data

5.1.7 Wallet Management

Before adding holdings, users set up wallets to define their investment goals (e.g., "Retirement," "Education," "Growth"). Each wallet has configurable profile settings—risk profile, investment horizon, objective, liquidity preference, and maximum drawdown—allowing distinct strategies within one account. Wallets are displayed as cards showing purpose, invested value, market value, and P/L.

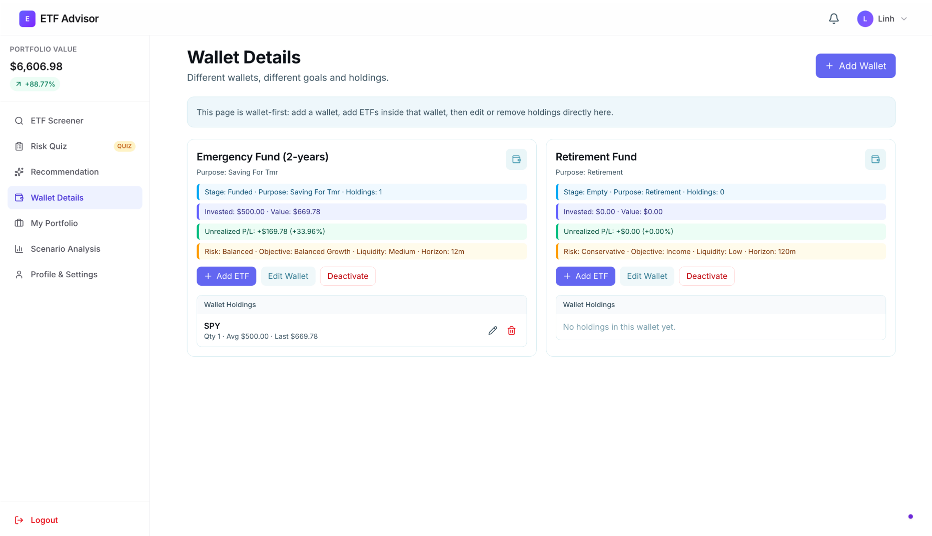

Figure 5.7: Wallet management with purpose-based organisation and per-wallet profiles

5.1.8 Portfolio Management

With wallets in place, users add ETF holdings via the “Add to Portfolio” button on any detail page or wallet page. The modal collects the ticker, quantity, purchase price, date, and target wallet. The portfolio overview aggregates all holdings and displays total market value, invested capital, unrealised P/L, a growth chart, an allocation pie chart, and a holdings table with inline editing. A wallet filter scopes the view to a specific wallet, and a news section surfaces articles for held tickers.

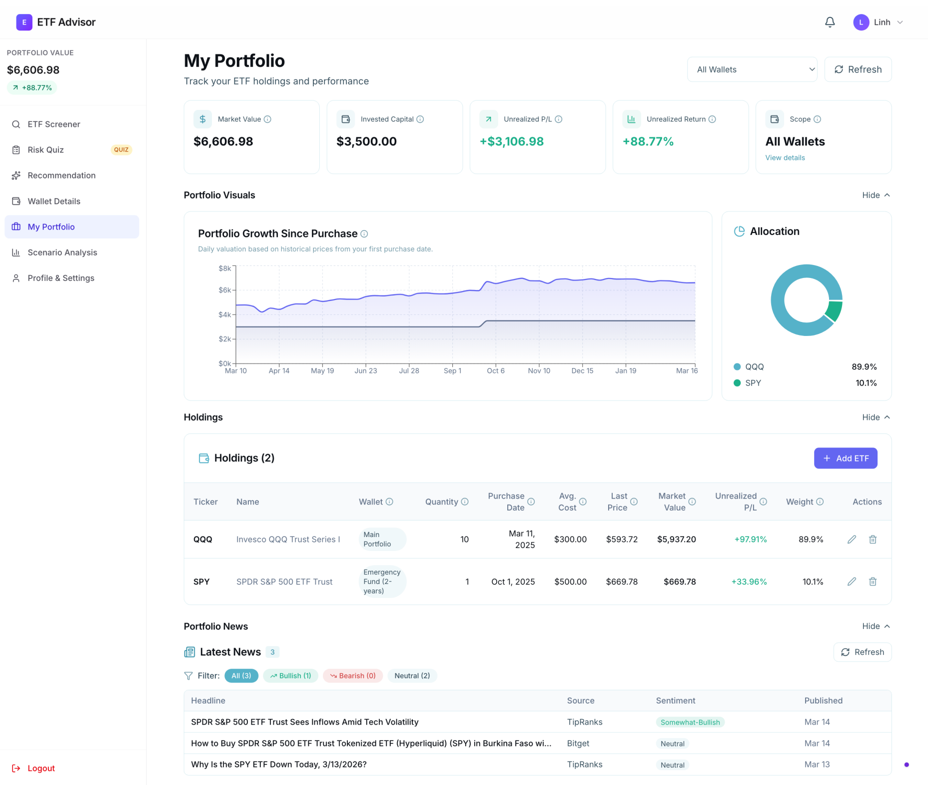

Figure 5.8: Portfolio overview with summary, growth chart, allocation, and holdings

5.1.9 Scenario Analysis

The Scenario Analysis page lets users stress-test holdings at the portfolio, wallet, or single-ETF level. Available scenarios include a COVID-19 crash replay (2020-02-19 to 2020-03-23) and uniform drawdowns (−10%, −20%, −30%). Results show baseline versus scenario values, percentage change, and per-holding impact with worst/best performer insights.

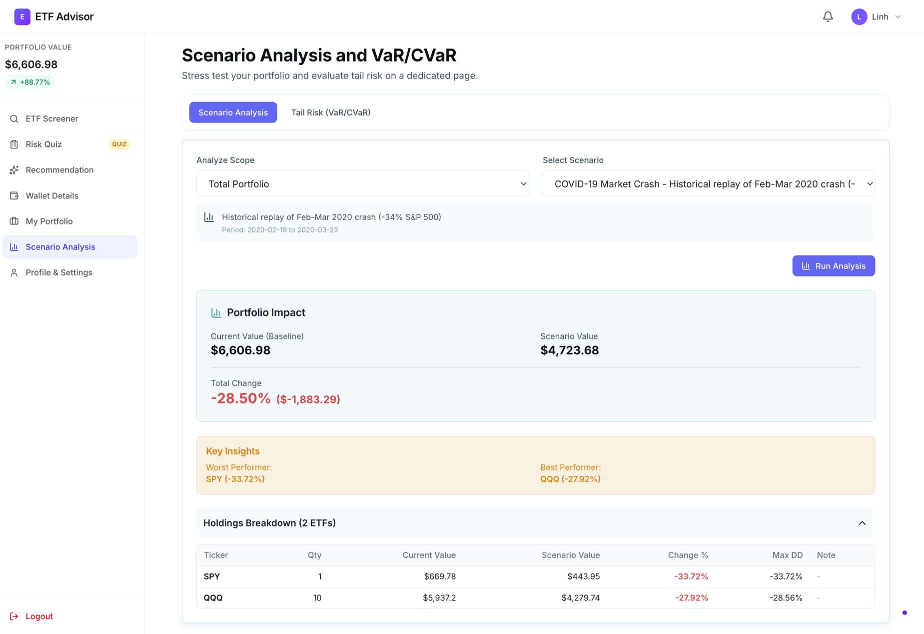

Figure 5.9: COVID-19 scenario analysis with per-holding impact breakdown

5.1.10 VaR/CVaR Tail Risk Analysis

A second tab provides VaR and CVaR at configurable confidence levels (90%, 95%, 99%) and time horizons (1M, 3M, 1Y, 2Y) using both historical and parametric methods. Results include dollar-amount risk estimates and portfolio statistics (mean daily return, volatility, worst/best day).

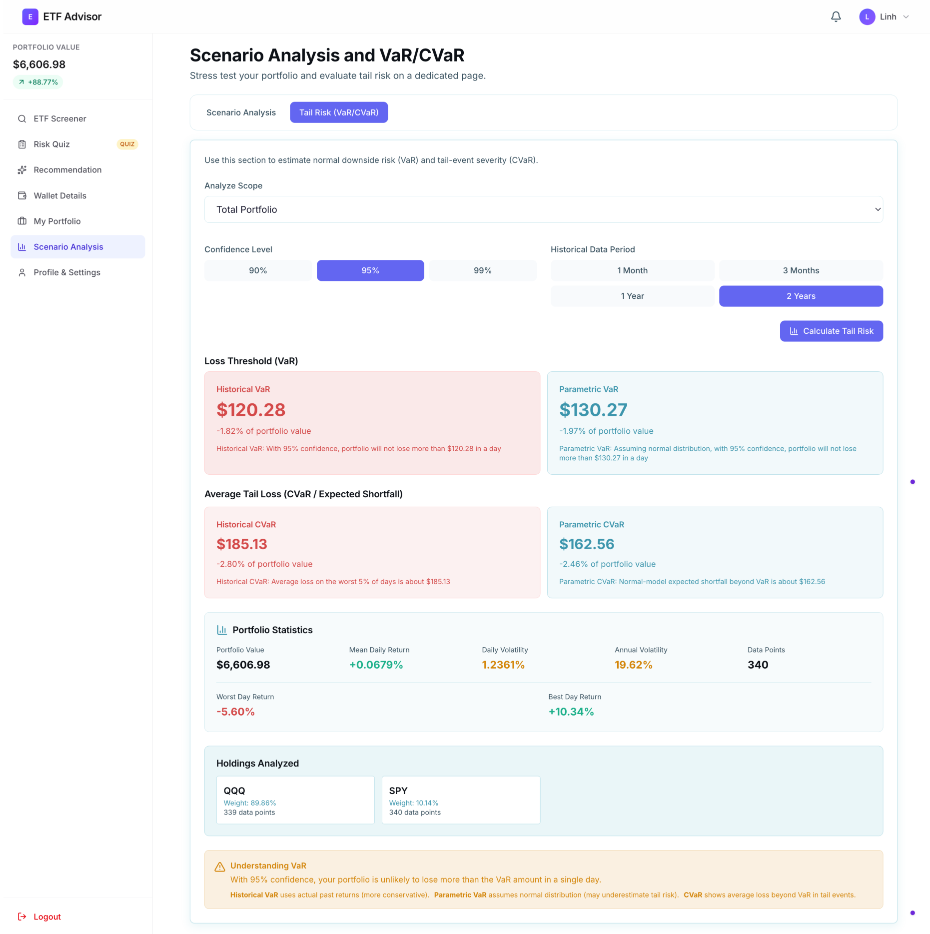

Figure 5.10: VaR/CVaR tail risk analysis with historical and parametric estimates

5.2 Testing and Evaluation

Having demonstrated the platform's features, this section presents the testing methodology and results that validate system correctness, performance, and usability. Backend tests were executed using FastAPI TestClient and Swagger UI against a development instance connected to the Supabase PostgreSQL database.

5.2.1 API Testing

The backend exposes 49 endpoints in total (47 router endpoints across 12 modules and 2 system endpoints in `main.py`: `/` and `/health`). A full API validation cycle was executed across all endpoints, with 124 consolidated test cases covering success paths, input validation, authentication enforcement, and error handling.

Module	Endpoints	Test Cases	Pass Rate
Authentication	4	14	100%
ETF Data (etfs + prices)	7	18	100%
Metrics Engine	3	10	100%
News	6	14	100%
Macro Indicators	4	10	100%
Risk Profiling (quiz)	3	10	100%
Recommendations	2	8	100%
Portfolio	5	14	100%
Wallet Management	8	16	100%
Scenario Analysis	4	6	100%
Chatbot	1	2	100%
System (root / health)	2	2	100%
Total	**49**	**124**	**100%**

Table 5.1: API Endpoint Test Coverage

Test cases included authentication checks (duplicate signup rejected with 400, missing token returns 401, cross-user wallet access returns 403), data retrieval validation (invalid ticker returns 404, paginated ETF list returns correct page size), input validation via Pydantic schemas (type mismatches and missing fields return 422), and computation verification (VaR/CVaR returns both historical and parametric values, scenario analysis returns per-holding impact).

5.2.2 Performance Evaluation

System performance was benchmarked against the non-functional requirements defined in Chapter 3.

Operation	Metric	Target	Result
ETF List Load (paginated)	Response Time	<500ms	120ms
ETF Detail with Metrics	Response Time	<500ms	280ms
Scenario Analysis (COVID)	Calculation Time	<3000ms	1200ms
VaR/CVaR (252-day, 95%)	Calculation Time	<3000ms	950ms
AI Chatbot	Response Generation	<5000ms	2500ms
Recommendation Generation	Response Time	<1000ms	350ms
Lighthouse Score	Performance	>80	88

Table 5.2: Performance Benchmarks

All operations met or exceeded their target thresholds. The chatbot response time is dominated by the external OpenAI API call latency rather than internal processing.

5.2.3 User Acceptance Testing (UAT)

A UAT session was conducted with 8 retail investors (age 18-22; 5 beginners, 3 intermediate) to evaluate usability. Participants completed seven tasks without assistance:

Task	Description	Completion	Avg. Time
T1	Register and complete the risk questionnaire	100%	2.5 min
T2	View personalised ETF recommendations	100%	1.0 min
T3	Search for a specific ETF and view its details	100%	1.5 min
T4	Add two ETF holdings to the portfolio	100%	2.0 min
T5	Create a wallet and assign a holding	100%	2.5 min
T6	Run a scenario analysis (COVID crash) on portfolio	87.5%	3.0 min
T7	Ask the AI chatbot a question about an ETF	100%	1.0 min

Table 5.3: UAT Task Completion Results

Users rated Ease of Navigation at 4.5/5, Usefulness of Recommendations at 4.3/5, Clarity of Risk Metrics at 4.1/5, Helpfulness of AI Chatbot at 4.4/5, and Overall Satisfaction at 4.4/5, validating the platform's core value proposition.

Chapter 6

Conclusions and Future Work

6.1 Conclusions

This project set out to design and implement an AI-assisted ETF recommendation platform that helps retail investors build and manage diversified portfolios aligned with their risk preferences. Evaluating the project against the objectives defined in Chapter 1, the following outcomes were achieved:

1. **Platform design and implementation.** A full-stack web application was delivered using Next.js 14 (frontend) and FastAPI (backend), deployed on Vercel and Render respectively, with a Supabase-hosted PostgreSQL database. The architecture cleanly separates presentation, application, and data layers, enabling independent scaling and maintenance.
2. **Rule-based recommendation engine.** A multi-factor scoring engine was implemented that filters and ranks ETFs according to the user's risk profile (conservative, balanced, or aggressive). Scoring signals include beta alignment, return and dividend heuristics, expense ratio, and category matching, with each recommendation accompanied by human-readable explanations.
3. **AI-powered educational chatbot.** An OpenAI-integrated chatbot was deployed on every ETF detail page. Retrieval-style grounding, where live ETF metadata and computed metrics are injected into the prompt context, reduces hallucination risk and keeps responses factually anchored. A financial disclaimer is included in every response.

4. **Data aggregation and metrics computation.** The platform ingests data from five external sources (Yahoo Finance, ETFdb, Alpha Vantage, FRED, and OpenAI), covering 2,272 ETFs and over 5.7 million historical OHLCV records. Key financial metrics, including Sharpe ratio, beta, volatility, and maximum drawdown, are computed on demand using vectorized Pandas operations.
5. **Risk analysis tools.** Historical scenario replay (e.g., the COVID-19 crash of February–March 2020), uniform stress tests (-10%, -20%, -30%), and both historical and parametric VaR/CVaR at configurable confidence levels (90%, 95%, 99%) were implemented and validated against target response times.
6. **Wallet-level portfolio organization.** A purpose-based wallet system allows users to group holdings by investment objective while maintaining separate risk profiles, horizons, and valuation summaries per wallet. Scenario analysis and VaR can be scoped to a single ETF, a wallet, or the full portfolio.
7. **Responsive user interface.** The frontend was verified across mobile, tablet, and desktop viewports and achieved Lighthouse scores of 88 (Performance), 91 (Accessibility), 95 (Best Practices), and 92 (SEO).
8. **Testing and user validation.** A total of 124 API test cases across 49 endpoints achieved a 100% pass rate. User acceptance testing with 8 participants yielded an overall satisfaction score of 4.4/5, confirming the platform's usability and educational value.

Taken together, these results demonstrate that the platform fulfils its stated objectives: providing professional-grade ETF analytics, risk-aware recommendations, and AI-assisted education in a single, accessible interface for retail investors.

6.2 Limitations

Despite successful implementation, several technical and functional constraints remain.

Market and Data Coverage

- The platform covers only U.S.-listed and Singapore-listed ETFs (2,272 funds), excluding European UCITS ETFs, Hong Kong-listed funds, and other emerging-market products. Expanding coverage requires onboarding new data providers, multi-currency normalization, and handling different trading calendars.

- Prices and news follow scheduled batch refresh cycles (daily and monthly) rather than real-time streaming, so intraday movements and breaking events are not immediately reflected. Moving to near real-time would require WebSocket-based data feeds and significant pipeline changes.

Analytical Constraints

- The recommendation engine uses rule-based scoring with static thresholds and weights. While transparent and explainable, it cannot learn from historical outcomes or adapt to evolving market regimes unlike machine-learning-based approaches.

Platform Functionality

- The wallet workflow is planning-oriented only; there is no broker API integration, order routing, or real-time position synchronization, so users must execute trades through external platforms.
- The platform does not include collaborative features such as shared portfolios, peer comparison, or community discussion.

AI Chatbot

- The chatbot depends entirely on the OpenAI API, introducing response latency (averaging 2.5 seconds), ongoing operational costs, and a dependency on third-party availability and pricing changes.

6.3 Lessons Learned

The development process yielded several practical insights for building data-intensive financial web applications.

- **Data quality is the foundation.** Investing early in a robust preprocessing pipeline (gap filling, deduplication, format normalization) prevented downstream errors in charts, metrics, and VaR calculations that would have been far harder to diagnose later.
- **Design around API rate limits from the start.** Alpha Vantage and Yahoo Finance throttling made naïve sequential refresh infeasible. Proactive decisions—

AUM-based prioritization, batch downloads of 50 tickers, and 24-hour deduplication caches—had to be embedded in the architecture upfront rather than retrofitted.

- **Explainability drives user trust.** UAT participants consistently valued human-readable reason labels (e.g., "Very low volatility," "Low expense ratio") over raw scores, reinforcing the importance of transparent, interpretable recommendations.
- **Modular architecture aids iteration.** Organizing the backend into distinct routers, services, and schemas per domain allowed features like wallet management and scenario analysis to be developed and tested in isolation, reducing integration friction.
- **Grounding reduces LLM hallucination.** Injecting verified ETF metadata into the chatbot's system prompt substantially improved factual accuracy, offering a lightweight alternative to full RAG pipelines for domain-specific applications.
- **Test with users early.** The UAT-identified navigation issue with the scenario analysis page was a straightforward fix that could have been caught sooner with lightweight usability reviews at earlier milestones.

6.4 Future Work

Future development aims to evolve the platform from an educational analytics tool into a broader investment support ecosystem. Planned enhancements are organized into three time horizons.

6.4.1 Short-Term Enhancements

- **Enhanced recommendation personalization:** incorporate user-selected ESG criteria, sector tilts, and dividend-income targets into the scoring engine.
- **Expanded market coverage:** add European UCITS ETFs, Hong Kong-listed funds, and other major markets with multi-currency normalization and cross-market trading calendar support.
- **Historical backtesting:** allow users to simulate how a wallet strategy would

have performed over a past period, providing empirical evidence for investment decisions.

6.4.2 Medium-Term Enhancements

- **ML-enhanced recommendations:** transition to a hybrid approach using supervised learning to optimize scoring weights while retaining explainable factor decomposition.
- **Near real-time data:** integrate WebSocket-based feeds or more frequent polling for intraday portfolio monitoring.

6.4.3 Long-Term Vision

- **Native mobile applications:** iOS and Android apps for on-the-go portfolio monitoring and push notifications.
- **Brokerage API integration:** connect with platforms such as Interactive Brokers or Tiger Brokers for one-click order placement, automated rebalancing, and real-time position synchronization.
- **Social and collaborative features:** portfolio sharing, peer comparison, and community discussion boards with privacy controls.
- **Advanced AI capabilities:** fine-tuned financial LLMs, automated weekly portfolio reports, and proactive risk alerts to evolve the chatbot into an intelligent investment co-pilot.

Together, these enhancements chart a path toward a comprehensive investment ecosystem that makes advanced ETF analytics and portfolio management progressively more accessible to retail investors.

References

[1] J. Chen. *Exchange-Traded Fund (ETF): What it is and how to invest*. Accessed January 2026. Dec. 2025. URL: https://www.investopedia.com/terms/e/etf.asp.

[2] *What is an ETF | iShares – BlackRock*. Accessed January 2026. BlackRock. URL: https://www.blackrock.com/sg/en/ishares/education/what-is-an-etf.

[3] J. Morgan. *ETF Market Guide: Trends and themes to watch*. Accessed January 2026. URL: https://www.jpmorgan.com/insights/global-research/investing/etf-guide.

[4] "Effects of information overload on financial markets: How much is too much?" In: (2024). URL: http://www.efmaefm.org/0EFMAMEETINGS/EFMA%20ANNUAL%20MEETINGS/2024-Lisbon/papers/paper_news2.pdf.

[5] Annamaria Lusardi and Olivia S. Mitchell. "The Economic Importance of Financial Literacy: Theory and Evidence". In: *Journal of Economic Literature* 52.1 (2014), pp. 5–44. URL: https://gflec.org/wp-content/uploads/2014/12/economic-importance-financial-literacy-theory-evidence.pdf.

[6] M. Seath and H. Gordon. *Retail Investors and ETFs*. Tech. rep. The Investment Association and Opinium Research, 2025. URL: https://www.theia.org/sites/default/files/2025-09/Retail%20Investors%20ETFs%20report_Final.pdf.

[7] *How SPY reinvented investing: The story of the first US ETF*. Jan. 2025. URL: https://www.ssga.com/us/en/intermediary/insights/how-spy-reinvented-investing-story-of-first-us-etf.

[8] T. Segal. *Measuring a Portfolio's Performance*. May 2025. URL: https://www.investopedia.com/articles/08/performance-measure.asp.

[9] William F. Sharpe. "Mutual Fund Performance". In: *The Journal of Business* 39.1 (1966), pp. 119–138. URL: https://www.jstor.org/stable/2351741.

[10] H. Markowitz and The Rand Corporation. "Portfolio Selection". In: *The Journal of Finance* 7.1 (1952), pp. 77–91. URL: http://www.gacetafinanciera.com/TEORIARIESGO/MPS.pdf.

[11] William F. Sharpe. "Capital Asset Prices: A Theory of Market Equilibrium under Conditions of Risk". In: *The Journal of Finance* 19.3 (1964), pp. 425–442. DOI: 10.1111/j.1540-6261.1964.tb02865.x.

[12] Philippe Jorion. *Value at Risk: The New Benchmark for Managing Financial Risk*. 3rd ed. McGraw-Hill, 2006. ISBN: 978-0071464956.

[13] R. Tyrrell Rockafellar and Stanislav Uryasev. "Optimization of Conditional Value-at-Risk". In: *Journal of Risk* 2.3 (2000), pp. 21–41. DOI: 10.21314/JOR.2000.038.

[14] Eugene F. Fama. "Efficient Capital Markets: A Review of Theory and Empirical Work". In: *The Journal of Finance* 25.2 (1970), pp. 383–417. DOI: 10.2307/2325486.

[15] Tim Loughran and Bill McDonald. "When Is a Liability Not a Liability? Textual Analysis, Dictionaries, and 10-Ks". In: *The Journal of Finance* 66.1 (2011), pp. 35–65. DOI: 10.1111/j.1540-6261.2010.01625.x.

[16] Francesco Ricci, Lior Rokach, and Bracha Shapira. *Recommender Systems Handbook*. 2nd ed. Springer, 2015. DOI: 10.1007/978-1-4899-7637-6.

[17] G. Bagattini et al. *Leveraging Large Language Models in Finance: Pathways to Responsible Adoption*. ESMA and Turing Institute. URL: https://www.esma.europa.eu/sites/default/files/2025-06/LLMs_in_finance_-_ILB_ESMA_Turing_Report.pdf.

[18] Patrick Lewis et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 33. 2020, pp. 9459–9474.

[19] S. Shehmir and R. Kashef. “LLM4REC: A comprehensive survey on the integration of large language models in Recommender Systems—Approaches, Applications and Challenges”. In: *Future Internet* 17.6 (2025), p. 252. DOI: 10.3390/fi17060252.

Appendix A

Use Case Specifications

A.1 UC-01: Register Account

Use Case ID	UC-01
Use Case Name	Register Account
Primary Actor	Retail Investor
Description	Enables a new user to create an account on the platform by providing valid registration credentials.
Preconditions	• User has access to the platform via a supported web browser. • User does not already have an account associated with the provided email address.

Main Flow	1. User navigates to the registration page. 2. User enters a valid email address. 3. User enters a password with a minimum length of six characters. 4. User confirms the password. 5. User submits the registration form. 6. System validates the input fields. 7. System creates a new user account. 8. System authenticates the user and redirects them to the risk questionnaire.
Alternative Flows	**AF-1: Email Already Registered** At Step 6, if the email already exists, the system displays an error message. **AF-2: Invalid Input** If validation fails, the system highlights invalid fields and prompts correction.
Postconditions	• User account created and authenticated via JWT.
Business Rules	• Password must be $\geq 6 charactersandsecurelyhashed.$

A.2 UC-02: Login / Authenticate

Use Case ID	UC-02
Use Case Name	Login / Authenticate
Primary Actor	Retail Investor
Description	Allows an existing user to log in to the platform using registered credentials.
Preconditions	• User has a registered account.

Main Flow	1. User navigates to the login page. 2. User enters email and password. 3. User submits login request. 4. System validates credentials. 5. System issues authentication token (JWT). 6. System redirects user to dashboard.
Alternative Flows	**AF-1: Invalid Credentials** If credentials are incorrect, system displays an authentication error.
Postconditions	• User is authenticated and logged in.
Business Rules	• Tokens expire after a predefined duration.

A.3 UC-03: Take Risk Questionnaire

Use Case ID	UC-03
Use Case Name	Take Risk Questionnaire
Primary Actor	Retail Investor
Description	User completes a psychometric questionnaire to determine their investment risk profile.
Preconditions	• User is authenticated.
Main Flow	1. User navigates to questionnaire page. 2. System displays 8 multiple-choice questions. 3. User selects answers and submits. 4. System calculates total score. 5. System assigns risk profile (Conservative / Balanced / Aggressive).
Postconditions	• User risk profile is saved for recommendations.

Business Rules	**Score Classification:** Thresholds are loaded from quiz configuration and mapped to Conservative / Balanced / Aggressive profiles.

A.4 UC-04: Search and Filter ETFs

Use Case ID	UC-04
Use Case Name	Search & Filter ETFs
Primary Actor	Retail Investor
Description	Allows user to search and filter ETFs by name, sector, risk profile, or other metrics.
Preconditions	• User has access to ETF discovery page.
Main Flow	1. User navigates to the ETF search page. 2. User enters search criteria or selects filters. 3. System applies filters and displays results.
Alternative Flows	**AF-1: No Results Found** System displays "No ETFs found matching your criteria."
Postconditions	• Filtered ETF list is displayed to the user.
Business Rules	• Filters must respect user risk profile and platform limits.

A.5 UC-05: View ETF Details & Charts

Use Case ID	UC-05
Use Case Name	View ETF Details & Charts
Primary Actor	Retail Investor
Description	User views detailed information and performance charts for a selected ETF.

Preconditions	• User has access to ETF detail page. • ETF selected from search or recommendations.
Main Flow	1. User selects an ETF. 2. System retrieves ETF data (price history, risk metrics). 3. System displays detailed info and charts.
Alternative Flows	**AF-1: Data Unavailable** System shows "ETF data not available" and logs error.
Postconditions	• User can review ETF metrics and charts.
Business Rules	• Charts must reflect accurate historical data.

A.6 UC-06: Get Recommendations

Use Case ID	UC-06
Use Case Name	Get Recommendations
Primary Actor	Retail Investor
Description	System provides ETF recommendations based on user risk profile and sentiment analysis.
Preconditions	• User has selected a risk profile (from quiz result or manual profile selection).
Main Flow	1. User navigates to "Recommendations" page. 2. System retrieves selected risk profile and relevant ETF attributes. 3. System generates and displays ETF recommendations.
Alternative Flows	**AF-1: No Recommendations** System displays "No suitable ETFs available at this time."
Postconditions	• User can view recommended ETFs.

Business Rules	• Recommendations must always match risk profile constraints and rule-based scoring logic.

A.7 UC-07: Manage Portfolio

Use Case ID	UC-07
Use Case Name	Manage Portfolio
Primary Actor	Retail Investor
Description	User can add, remove, or update ETFs in their portfolio and track performance.
Preconditions	• User is authenticated.
Main Flow	1. User accesses portfolio page. 2. User adds/removes ETFs or updates holdings. 3. System recalculates portfolio performance. 4. System displays updated metrics and charts.
Alternative Flows	**AF-1: Invalid Entry** System prompts correction for invalid ETF or holding data.
Postconditions	• Portfolio reflects updated ETFs and performance metrics.
Business Rules	• Portfolio calculations must respect all user holdings and risk constraints.

A.8 UC-08: Run Scenario Analysis

Use Case ID	UC-08
Use Case Name	Run Scenario Analysis
Primary Actor	Retail Investor

Description	User runs stress testing and VaR analysis on portfolio, wallet, or selected ETF holdings.
Preconditions	• User is authenticated. • At least one analysis target exists (portfolio, wallet, or ETF).
Main Flow	1. User opens the scenario analysis page. 2. User selects analysis target type (portfolio, wallet, or ETF). 3. User selects a scenario (e.g., COVID crash or stress-10/stress-20/stress-30). 4. System simulates performance and returns impact metrics. 5. User optionally runs VaR/CVaR for the selected target.
Alternative Flows	**AF-1: Missing Data** System interpolates missing historical data or flags errors.
Postconditions	• User receives scenario impact and risk results for the selected target.
Business Rules	• Scenario analysis uses available historical price data and selected target holdings.

A.9 UC-11: Manage Wallets

Use Case ID	UC-11
Use Case Name	Manage Wallets
Primary Actor	Retail Investor
Description	User creates, updates, and deactivates goal-based wallets for organizing investments.
Preconditions	• User is authenticated.

Main Flow	1. User opens the wallet management page. 2. User creates a new wallet by entering wallet name and purpose. 3. System creates the wallet and initializes default wallet profile settings. 4. User optionally updates wallet profile fields (risk profile, horizon, objective, and liquidity settings). 5. User may deactivate an existing wallet.
Alternative Flows	**AF-1: Invalid Wallet Input** System prompts user to correct invalid values (for example name too short or invalid profile ranges).
Postconditions	• Wallet records and wallet profile settings are updated for the authenticated user.
Business Rules	• Wallet operations are user-scoped; users can only manage their own wallets. • Deactivation marks wallet as inactive rather than deleting historical data.

A.10 UC-12: Assign Holdings to Wallet

Use Case ID	UC-12
Use Case Name	Assign Holdings to Wallet
Primary Actor	Retail Investor
Description	User assigns a portfolio holding to a selected wallet for goal-based grouping and analytics.
Preconditions	• User is authenticated. • User has at least one existing wallet. • User has at least one portfolio holding.

Main Flow	1. User selects an existing holding from portfolio or wallet interface. 2. User chooses a target wallet. 3. System updates wallet assignment for the holding. 4. System refreshes wallet holdings and portfolio summaries.
Alternative Flows	**AF-1: Wallet Not Found or Inactive** System returns an error and asks user to select an active wallet.
Postconditions	• Holding is mapped to the selected wallet and reflected in wallet-level views.
Business Rules	• A holding can be mapped to one active wallet assignment at a time. • Assignment changes trigger recalculation of wallet-level current value and allocation outputs.

A.11 UC-09: Ask AI Chatbot

Use Case ID	UC-09
Use Case Name	Ask AI Chatbot
Primary Actor	Retail Investor
Description	User asks questions about ETFs, portfolios, or investment guidance and receives responses from the AI chatbot.
Preconditions	• User is authenticated. • Chatbot service is online.
Main Flow	1. User inputs question into chatbot interface. 2. System processes query using AI model. 3. System returns answer or advice to user.

Alternative Flows	**AF-1: Query Not Understood** System responds: "I'm sorry, I didn't understand your question."
Postconditions	• User receives AI response or guidance.
Business Rules	• Responses must comply with platform guidelines and factual accuracy.

A.12 UC-10: View News Sentiment

Use Case ID	UC-10
Use Case Name	View News Sentiment
Primary Actor	Retail Investor
Description	User views ETF and portfolio-related news with sentiment labels and alerts.
Preconditions	• News data source is available. • User has selected a ticker or portfolio context.
Main Flow	1. User opens a ticker detail page or portfolio dashboard. 2. System retrieves recent related news articles. 3. System displays sentiment labels and relevance details. 4. User reviews alerts and article links.
Alternative Flows	**AF-1: API Failure** System retries up to 3 times, then logs error.
Postconditions	• User sees current sentiment-tagged news for decision support.
Business Rules	• News windows are query-driven (for example last N days), not fixed to a single 24-hour window. • Sentiment scores normalized between -1 and +1.

Appendix B

Risk Questionnaire List

This appendix documents the questionnaire used to estimate user risk tolerance and assign recommendation profiles. The ETF Goal & Risk Quick Check includes 8 multiple-choice questions (A–D), with responses scored and summed to classify users as Conservative, Balanced, or Aggressive. The backend computes the total from selected answers (skipping unanswered ones, though the frontend requires all questions), with a configured maximum of 720 and an effective maximum of 715. Profile thresholds are defined in the backend and can be adjusted without changing API logic. This tool supports recommendation personalization and does not constitute regulated investment advice.

B.1 Question Set (8 Questions)

QID	Question	Options (Points)
Q1	When do you expect to need this money?	A: Within 3 months (5) B: 3–12 months (20) C: 1–3 years (55) D: More than 3 years (90)

QID	Question	Options (Points)
Q2	What is your primary purpose for this investment?	A: Protect my principal with minimal fluctuation (10) B: Beat bank savings rate with low risk (25) C: Balanced growth over time (60) D: Maximize long-term growth (90)
Q3	If your ETF position drops 8% in one month, what would you likely do?	A: Sell immediately to avoid further loss (10) B: Reduce part of my position (35) C: Hold and wait (65) D: Add more if fundamentals still look good (90)
Q4	How much downside can you accept before this investment causes real stress?	A: About 2% or less (10) B: Around 5% (35) C: Around 10% (65) D: 15% or more (90)
Q5	Do you already have an emergency fund (at least 3–6 months of expenses)?	A: No (10) B: Partially (1–3 months) (35) C: Yes (3–6 months) (65) D: Yes (more than 6 months) (85)
Q6	Which best describes this money amount?	A: Critical cash I may need soon (10) B: Important, but not urgent (35) C: Dedicated investment capital (65) D: High-risk growth capital I can leave untouched (90)

QID	Question	Options (Points)
Q7	How familiar are you with ETFs and market behavior?	A: Beginner (10) B: Basic understanding (35) C: Comfortable selecting broad ETFs (65) D: Advanced and comfortable with volatility (90)
Q8	What return-vs-stability tradeoff fits you best?	A: I prefer stable value even if returns are lower (10) B: Modest outperformance vs savings with low swings (30) C: Moderate fluctuations for better growth (65) D: Prioritize growth with higher volatility (90)

Table B.1: ETF Goal & Risk Quick Check Questionnaire

B.2 Profile Thresholds

Profile	Min Score	Max Score	Description
Conservative	0	260	Low-risk investor prioritizing capital preservation and stable income
Balanced	261	500	Moderate-risk investor balancing growth with stability
Aggressive	501	720	High-risk investor prioritizing capital growth over stability

Table B.2: Risk Profile Mapping for the 8-Question Questionnaire